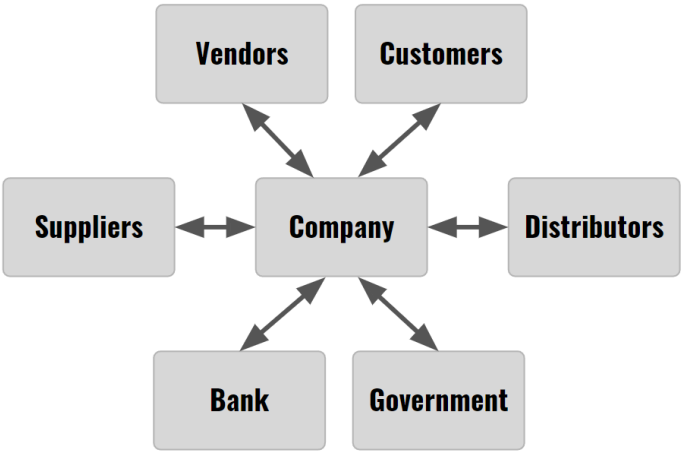
Developing Full Stack Next.js Web Applications

Dr. Jose Annunziato

Table of Contents

Chapter 5 - Implementing RESTful Web APIs with Express.js

During the 90s, the adoption of the World Wide Web grew exponentially. A variety of commercial ventures explored numerous use cases, revolutionizing interactions between companies, their customers, and other businesses. The figure on the right illustrates several integration points between businesses, often referred to as ***business-to-business*** (***B2B***) interactions. Interactions between businesses and customers are commonly known as ***business-to-consumer*** (***B2C***) interactions. Many companies have largely automated customer interactions by implementing online storefronts where customers can browse products, place orders, submit reviews, and process returns.

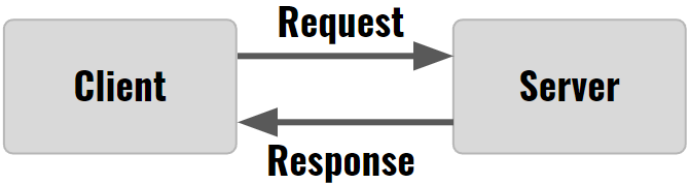


Creating visually appealing ***user interfaces*** is essential to capture customer attention, encourage purchases through marketing ads, and build long-term relationships through incentives like discounts and loyalty programs. User interfaces, as the name suggests, focus on application aspects that interact with users through visually engaging representations of data. Up to this point, these interfaces have used hard-coded JSON files, such as ***courses.json*** and ***modules.json***, to render data. Interfaces have been built to render and manipulate this data, updating the screen to reflect changes.

However, these updates have not been permanent; refreshing the browser results in lost changes and a reset application state. JavaScript applications running on clients like browsers, game consoles, or TV boxes have limited options for retrieving and storing data permanently. The next chapters address the challenges of retrieving, storing, updating, and deleting data permanently on remote servers and databases from React applications.

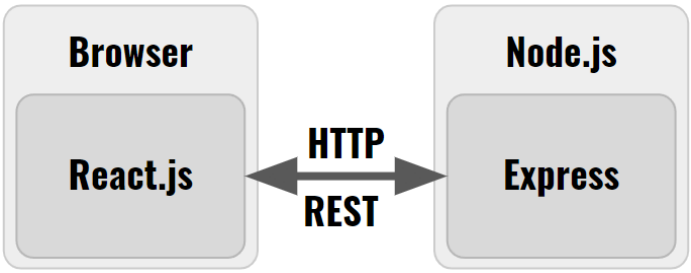
5.1 Installing and Configuring an HTTP Web Server

The Kambaz React Web application built so far is the ***client*** in a ***client/server architecture***. Users interact with client applications that implement user interfaces relying on servers to store data and execute complex logic that would be impractical on the client. Clients and servers interact through a sequence of requests and responses. Clients send ***requests*** to servers, servers execute some logic, fulfill the request, and then ***respond*** back to the client with results. This section discusses implementing HTTP servers using Node.js.



5.1.1 Introduction to Node.js

JavaScript is generally recognized as a programming language designed to execute in browsers, but it has been rescued from its browser confines by Node.js. Node.js is a JavaScript runtime to interpret and execute applications written in JavaScript outside of browsers, such as from a desktop console or terminal. This allows JavaScript applications written for the desktop to overcome many limitations faced by those in the browser. JavaScript running in a browser is restricted, with no access to the filesystem or databases, and limited network capabilities. In contrast,



JavaScript running on a desktop has full access to the filesystem, databases, and unrestricted network access. Conversely, desktop JavaScript applications generally lack a user interface and offer limited user interaction, while browser-based JavaScript applications provide rich and sophisticated interfaces for user interaction.

5.1.2 Installing Node.js

Node.js is a JavaScript runtime that can execute JavaScript on a desktop, allowing JavaScript programs to breakout from the confines and limitations of a browser. Node.js was installed during previous chapters while implementing the React Web application. Confirm the installation and check the version by typing the following in your computer terminal or console application.

```
$ node  -v
v22.11.0
```

If a Node installation is present, its version will be displayed on the console; otherwise, an error message will be shown, indicating that Node.js needs to be downloaded and installed from the URL below. As of this writing, Node.js 22.11.0 was the latest version, but any version recommended on Node's website can be installed.

```
https://nodejs.org/en
```

Once downloaded, double click on the downloaded file to execute the installer, give the operating system all the permissions it requests, accept all the defaults, let the installer complete, and restart the computer. Once the computer is up and running again, confirm Node.js installed properly by running the command ***node -v*** again from the command line.

5.1.3 Creating a Node.js Project

Another tool installed along with Node.js is ***npm*** or ***Node Package Manager*** which has been used thus far to run React applications in previous chapters. The ***npm*** command can also be used to install and execute Node.js ***packages*** on the local computer as well as creating brand new Node.js projects. To create a Node.js project create a directory with the name of the desired project and then change into that directory as shown below. Choose a directory name that does not contain any spaces, is all lowercase, and uses dashes between words.

NOTE: ***DO NOT*** create the Node.js project directory inside the existing Next.js React project directory. The Next.js React project should be in a directory called ***kambaz-next-js*** (or similar) and the new ***kambaz-node-server-app*** directory ***SHOULD NOT*** be inside the Next.js React project directory. Instead, the two directories should be siblings, e.g., they should have the same parent directory.

```
$ mkdir  kambaz-node-server-app
$ cd     kambaz-node-server-app
```

AGAIN NOTE: ***DO NOT*** create the Node.js project directory inside the existing Next.js React project directory. The Next.js React project should be in a directory called ***kambaz-next-js*** (or similar) and the new ***kambaz-node-server-app*** directory ***SHOULD NOT*** be inside the Next.js React project directory. Instead, the two directories should be siblings, e.g., they should have the same parent directory.

Once in the Node.js project directory, use ***npm init*** to create a new Node.js project as shown below. This will kickoff an interactive session asking details about the project such as the name of the project and the author. The following is a sample interaction with sample answers. Each question provides a default answer which can be accepted or skipped by just pressing enter. It is fine to initially keep all the default values since they can be configured later.

```
$ npm  init
package name: (kambaz-node-server-app)
version: (1.0.0)
description: Node.js HTTP Web server for the Kambaz application
entry point: (index.js)
test command:
git repository: https://github.com/jannunzi/kambaz-node-server-app
keywords: Node, REST, Server, HTTP, Web Development, CS5610, CS4550
author: Jose Annunziato
license: (ISC)
```

The configuration will be written into a new file called ***package.json*** in a JSON format and it's distinctive of Node.js projects, like ***pom.xml*** files might be distinctive for Java projects.

5.1.4 Creating a Simple Hello World Node.js Program

Open the Node.js project directory created earlier with an IDE such as Visual Studio Code or IntelliJ, and at the root of the project, create a JavaScript file called ***Hello.js*** with the content shown below. The script uses the ***console.log()*** function to print the string ***'Hello World!'*** to the console and it is a common first program to write when learning a new language or infrastructure.

Hello.js

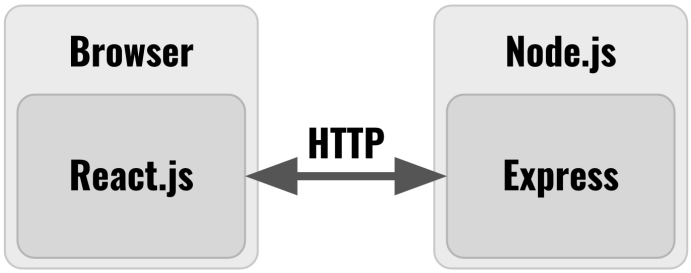
```
console.log("Hello World!");
```

At the command line, run the ***Hello.js*** application by using the ***node*** command and confirm the application prints ***Hello World!*** to the console as shown below.

```
$ node  Hello.js
Hello World!
```

Node.js programs consist of JavaScript files that are executed with the ***node*** command line interpreter. The following sections describe writing JavaScript applications that implement ***HTTP Web servers*** and ***RESTful Web APIs*** to integrate with React user interfaces. Upcoming chapters describe writing JavaScript applications that store and retrieve data from databases such as ***MongoDB***.

5.1.5 Creating a Node.js HTTP Web Server



Express is a very popular Node.js library that simplifies creating HTTP servers and Web APIs. Express HTTP servers can respond to HTTP requests from HTTP clients such as the React user interface implemented in earlier chapters. From the root directory of the Node.js project, install the ***express*** library from the terminal as shown below.

```
npm  install  express
```

Confirm that an ***express*** entry appears in ***package.json*** in the ***dependencies*** property. It is important these dependencies are listed in ***package.json*** so that they can be re-installed by other colleagues or when deploying to remote servers and cloud platforms such as ***AWS***, ***Heroku***, or ***Render.js***. New libraries are installed in a new folder called ***node_modules***. More Node.js packages can be found at npmjs.com. The following ***index.js*** implements an HTTP server that responds ***Hello***

World! when the server receives an HTTP request at the URL ***http://localhost:4000/hello***. Copy and paste the URL in a browser to send the HTTP request and the browser will render the response from the server. The ***require*** function is equivalent to the ***import*** keyword and loads a library into the local source. The ***express()*** function call creates an instance of the express library and assigns it to local constant ***app***. The ***app*** instance is used to configure the server on what to do when various types of requests are received. For instance the example below uses the ***app.get()*** function to configure an ***HTTP GET Request handler*** by mapping the URL pattern ***'/hello'*** to a function that handles the HTTP request.

index.js ***s***

```
const express = require('express')                          // equivalent to import
const app = express()                                       // create new express instance
app.get('/hello', (req, res) => {res.send('Hello World!')})  // create a route that responds 'hello'
app.listen(4000)                                            // listen to http://localhost:4000
```

A request to URL ***http://localhost:4000/hello*** triggers the function implemented in the second argument of ***app.get()***. The handler function receives parameters ***req*** and ***res*** which allows the function to participate in the ***request/response*** interaction, common in ***client/server*** applications. The ***res.send()*** function responds to the request with the text ***Hello World!*** Use ***node*** to run the server from the root of the project as shown below.

```
$ node  index.js
```

The application will run, start the server, and wait at port ***4000*** for incoming HTTP requests. Point your browser to ***http://localhost:4000/hello*** and confirm the server responds with ***Hello World!*** Stop the server by pressing ***Ctrl+C***. The string ***http://localhost:4000/hello*** is referred to as a ***URL*** (***Uniform Resource Locator***) and is used to .

5.1.6 Configuring Nodemon to Automatically Restart Node.js Server

React Web applications automatically transpile and restart every time code changes. Node.js can be configured to behave the same way by installing a tool called ***nodemon*** which monitors file changes and automatically restarts the Node application. Install nodemon globally (***-g***) as follows.

```
$ npm  install  nodemon  -g
```

On ***macOS*** you might need to run the command as a ***super user*** as follows. Type your password when prompted.

```
$ sudo  npm  install  nodemon  -g
```

Now instead of using the node command to start the server, use ***nodemon*** as follows:

```
$ nodemon  index.js
```

Confirm the server is still responding ***Hello World!***. Change the response string to ***Life is good!*** and without stopping and restarting the server, refresh the browser and confirm that the server now responds with the new string. To practice, create another endpoint mapped to the root of the application, e.g., ***"/"***. Navigate to ***http://localhost:4000*** with your browser and confirm the server responds with the message below.

index.js ***s***

```
const express = require('express')
const app = express()
app.get('/hello', (req, res) => {res.send('Life is good!')})   // http://localhost:4000/hello responds "Life is good"
app.get('/', (req, res) => {                                   // http://localhost:4000 responds "Welcome to Full ..."
  res.send('Welcome to Full Stack Development!')})
```

```
app.listen(4000)
```

5.1.7 Configuring Node.js to Use ES6

The React Web application created in earlier chapters has used the ***import*** keyword to load ES6 modules, but note that the ***index.js*** is using the ***require*** keyword instead to accomplish the same thing. Since Node version 12, ES6 syntax is supported by configuring the ***package.json*** file and adding a new ***"type"*** property with value ***"module"*** as shown below in the highlighted text.

package.json

```
{
 "type": "module",                                    // type module turns on ES6
 "name": "kambaz-node-server-app",
 ...
 "scripts": {
  "test": "echo \"Error: no test specified\" && exit 1",
  "start": "node index.js"                            // use npm start to start server
 },
```

Now, instead of using ***require()*** to load libraries, the familiar ***import*** statement can be used instead. Here's the ***index.js*** refactored to use ***import*** instead of ***require***. Restart the server, refresh the browser and confirm that the server responds as expected.

index.js

```
import express from 'express';
const app = express();
app.get('/hello', (req, res) => {res.send('Life is good!')})
app.get('/', (req, res) => {res.send('Welcome to Full Stack Development!')})
app.listen(4000);
```

5.1.8 Creating HTTP Routes

The ***index.js*** file creates and configures an HTTP server listening for incoming HTTP requests. So far we've created a simple ***hello*** HTTP ***route*** that responds with a simple string. Throughout this and later chapters, we're going to create quite a few other HTTP routes, too many to define them all in ***index.js***. Instead, it is best practice to group routes into dedicated ***routing files*** that collect related HTTP routes. To illustrate this principle, move the routes defined in ***index.js*** to the ***Hello.js*** file created earlier as shown bellow.

index.js

```
import express from 'express';
const app = express();
app.get('/hello', (req, res) => {                     // move this to Hello.js
  res.send('Life is good!')
});
app.get('/', (req, res) => {
  res.send('Welcome to Full Stack Development!')
});
app.listen(4000);
```

Copy the routes to ***Hello.js*** as shown below.

Hello.js

```
console.log('Hello World!');                    // don't need this anymore
app.get('/hello', (req, res) => {               // moved here from index.js
   res.send('Life is good!')
});
app.get('/', (req, res) => {
   res.send('Welcome to Full Stack Development!')
});
```

In our case ***Hello.js*** handles HTTP requests for a ***hello*** greeting and responds with a friendly reply. We're not done though. Notice that ***Hello.js*** references ***app*** which is undefined in the file. The ***app*** variable represents an express instance which we would be tempted to create an instance in the file, but instead only one instance should be shared across all routes implemented per server instance. Instead of creating a new express instance, pass ***app*** as a parameter in a function we can import and invoke from ***index.js*** as shown below.

Hello.js S

```
export default function Hello(app) {            // function accepts app reference to express module
  app.get('/hello', (req, res) => {             // to create routes here. We could have used the new
    res.send('Life is good!')                   // arrow function syntax instead
  })
  app.get('/', (req, res) => {
    res.send('Welcome to Full Stack Development!')
  })
}
```

Import ***Hello.js*** and pass app to the function as shown below. Note the ***.js*** extension in the import statement. Test ***http://localhost:4000/hello*** from the browser and confirm the reply is still friendly.

index.js S

```
import express from 'express'
import Hello from "./Hello.js"                  // import Hello from Hello.js
const app = express()
Hello(app)                                      // pass app reference to Hello
app.listen(4000)
```

Although the ***Hello.js*** route implementation is perfectly fine, embedding the callback function declaration within the route definition can be a challenging syntax for some. An alternative (arguably better) syntax is to declare the callback functions on their own and then reference them in the route declarations as shown below.

Hello.js S

```
export default function Hello(app) {
  const sayHello = (req, res) => {
    res.send("Life is good!");
  };
  const sayWelcome = (req, res) => {
    res.send("Welcome to Full Stack Development!");
  };
  app.get("/hello", sayHello);
  app.get("/", sayWelcome);
}
```

5.2 Lab Exercises

The following are a set of exercises to practice creating and integrating with an HTTP server from a React Web application. In your server application, create and import file ***Lab5/index.js*** where we will be implementing several server side exercises. Create a route that welcomes users to Lab 5.

Lab5/index.js ***S***

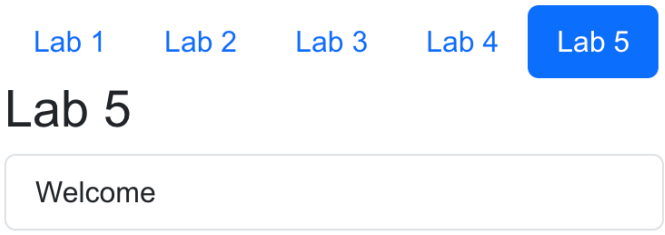
```
export default function Lab5(app) {                    // accept app reference to express module
  app.get("/lab5/welcome", (req, res) => {             // create route to welcome users to Lab 5.
    res.send("Welcome to Lab 5");                      // Here we are using the new arrow function syntax
  });
};
```

Import ***Lab5/index.js*** into the server ***index.js*** and pass it a reference to ***express*** as shown below. Restart the server and confirm that http://localhost:4000/lab5/welcome responds with the expected response.

index.js ***S***

```
import Hello from "./Hello.js";
import Lab5 from "./Lab5/index.js";                    // import Lab5
const app = express();
Lab5(app);                                             // pass reference to express module
Hello(app);
```

In the React Web app project, create a ***Lab5*** React component to test the Node HTTP server. Import the new component into the existing set of labs and add a new link in ***TOC.tsx*** to navigate to ***Lab5*** by selecting the corresponding tab or link as shown here on the right. The example below creates a hyperlink that navigates to the http://localhost:4000/lab5/ welcome URL. Confirm the link navigates to the expected response.



app/labs/lab5/page.tsx ***C***

```
export default function Lab5() {
  return (
    <div id="wd-lab5">
      <h2>Lab 5</h2>
      <div className="list-group">                     // hyperlink navigates to
        <a href="http://localhost:4000/lab5/welcome"   // http://Localhost:4000/Lab5/welcome
           className="list-group-item">
           Welcome
        </a>
      </div><hr/>
    </div>
);}
```

5.2.1 Environment Variables

Currently we are integrating the React user interface with a Node server that are both running locally on our development computers, but ultimately these will both be running on remote servers. Let's configure the local environment so that it will be easy later on to configure our source code to run in any environment. There are two environments to consider: the ***local development environment*** and the ***remote production environment***. The ***local development environment*** consists of our computer where we do our development running two Node servers, one hosting the React user interface Web application, and the other hosting the Express HTTP server. The ***remote production environments*** will consist of the React user interface running on Vercel, and the Express HTTP server running on ***Render.com***, ***Heroku***, or ***AWS*** (your choice). Environments can be configured with ***environment variables*** declared in your ***Operating System*** or as ***environment files*** in your project. Environment files are named ***.env*** (with a leading period) and can be defined for each environment by appending ***.development*** for the local development environment, ***.test*** for the test environment, and ***.production*** for the production environment. Instead of hardcoding ***http://localhost:4000*** everywhere in our source code, instead, declare an environment variable in the local environment file as shown below. Create the ***.env.development*** file at the root of your React project and copy the following content.

.env.development *c*

```
NEXT_PUBLIC_HTTP_SERVER=http://localhost:4000
```

In Next.js React projects, all environment variables must start with ***NEXT_PUBLIC_*** so that they are exported to the client components. Each line declares a variable, followed by an equal sign, followed by the value of the variable. Make sure not to use extra spaces or unnecessary extra characters such as quotes, commas, or colons. Every time a new variable is added, removed, or changes are made to an environment file, the React user interface Web application needs to be restarted. Environment variables can be accessed from the React source code through the global ***process*** object, in its ***env*** property. To instance, to access the value of the ***NEXT_PUBLIC_HTTP_SERVER*** declared above, use ***process.env.NEXT_PUBLIC_HTTP_SERVER***. To practice declaring and using environment variables, create component ***Environment Variables*** as shown below.

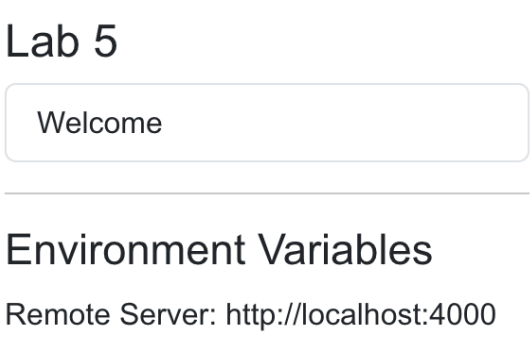
app/labs/lab5/EnvironmentVariables.tsx *c*

```
const HTTP_SERVER = process.env.NEXT_PUBLIC_HTTP_SERVER;
export default function EnvironmentVariables() {
  return (
    <div id="wd-environment-variables">
      <h3>Environment Variables</h3>
      <p>Remote Server: {HTTP_SERVER}</p><hr/>
    </div>
);}
```

Import the component in component ***Lab5***, restart the React application and confirm the URL of the remove server displays as shown below.

app/labs/lab5/page.tsx *c*

```
import EnvironmentVariables from "./EnvironmentVariables";
export default function Lab5() {
  return (
    <div id="wd-lab5">
      <h2>Lab 5</h2>
      <div className="list-group">
        <a href="http://localhost:4000/lab5/welcome"
           className="list-group-item">
           Welcome
        </a>
      </div><hr/>
      <EnvironmentVariables />
    </div>
);}
```



Make sure the URL to the remote server is never used in the React source code, instead prefer using the environment variable. Replace the ***http://localhost:4000*** in the previous exercise with the ***HTTP_SERVER*** constant as shown below. Confirm that the ***Welcome*** hyperlink still works.

app/labs/lab5/page.tsx *c*

```
import EnvironmentVariables from "./EnvironmentVariables";
const HTTP_SERVER = process.env.NEXT_PUBLIC_HTTP_SERVER;
export default function Lab5() {
  return (
    <div id="wd-lab5">
      <h2>Lab 5</h2>
      <div className="list-group">
        <a href={`${HTTP_SERVER}/lab5/welcome`} className="list-group-item">
          Welcome
        </a>
      </div><hr />
      <EnvironmentVariables />
    </div>
```

```
);}
```

Similarly, the Node Express server needs to be configured to run locally on your computer as well as when it is deployed in the remote environment. To do this, refactor ***index.js*** so that it uses the remote ***PORT*** environment variable if available, or port 4000 when running locally.

index.js ***S***

```
import express from 'express'
import Hello from "./Hello.js"
import Lab5 from "./Lab5/index.js";
const app = express()
Lab5(app)
Hello(app)
app.listen(process.env.PORT || 4000)
```

5.2.2 Sending Data to a Server via HTTP Requests

Let's explore how we can integrate the React user interface with the Node server by sending information to the server from the browser. There are three ways to send information to the server:

1. ***Path parameters*** - parameters are encoded as segments of the path itself, e.g., ***http://localhost:4000/lab5/add/2/5***.
2. ***Query parameters*** - parameters are encoded as name value pairs in the query string after the ? character at the end of a URL, e.g., ***http://localhost:4000/lab5/add?a=2&b=5***.
3. ***Request body*** - data is sent as a string representation of data encoded in some format such as XML or JSON containing properties and their values, e.g., ***{a:2, b: 5}*** or ***<params a=2 b=5/>***.

We'll explore the first two in this section, and address the last one towards the end of the labs.

5.2.2.1 Sending Data to a Server with Path Parameters

React applications can pass data to servers by embedding it in a URL path as ***path parameters*** part of a URL. For instance the last two integers -- 2 and 4 -- at the end of the following URL can be parsed by a corresponding matching route on the server, add the two integers, and respond with the result of 6.

```
http://localhost:4000/lab5/add/2/4
```

The following route declarations can parse path parameters a and b encoded in paths ***/lab5/add/:a/:b*** and ***/lab5/subtract/:a/:b***. In ***PathParameters.js***, implement the routes below and import it in ***Lab5/index.js***. On your own create routes ***/lab5/multiply/:a/:b*** and ***lab5/divide/:a/:b*** that calculate the arithmetic multiplication and division.

Lab5/PathParameters.js ***S***

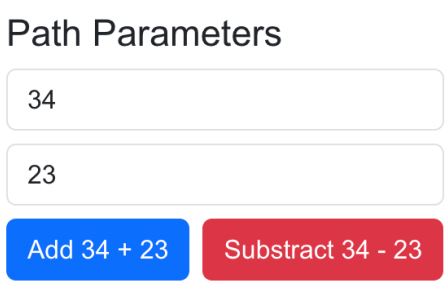
```
export default function PathParameters(app) {
 const add = (req, res) => {                    // route expects 2 path parameters after /lab5/add
   const { a, b } = req.params;                 // retrieve path parameters as strings
   const sum = parseInt(a) + parseInt(b);       // parse as integers and adds
   res.send(sum.toString());                    // sum as string sent back as response
 };                                             // don't send integers since can be interpreted as status
 const substract = (req, res) => {              // route expects 2 path parameters after /lab5/subtract
   const { a, b } = req.params;                 // retrieve path parameters as strings
   const sum = parseInt(a) - parseInt(b);       // parse as integers and subtracts
   res.send(sum.toString());                    // subtraction as string sent back as response
 };                                             // response is converted to string otherwise browser
```

```
 app.get("/lab5/add/:a/:b", add);                    // would interpret integer response as a status code
 app.get("/lab5/subtract/:a/:b", substract);
};
```

Note when you import, make sure to include the extension ***.js*** as shown below. Pass a reference of ***app*** to the ***Path Parameters*** function. Confirm that http://localhost:4000/lab5/add/6/4 responds with 10 and http://localhost:4000/lab5/subtract/6/4 responds with 2. Also confirm the routes you implemented on your own.

Lab5/index.js ***S***

```
import PathParameters from "./PathParameters.js";
export default function Lab5(app) {
  app.get("/lab5/welcome", (req, res) => {
    res.send("Welcome to Lab 5");
  });
  PathParameters(app);
}
```

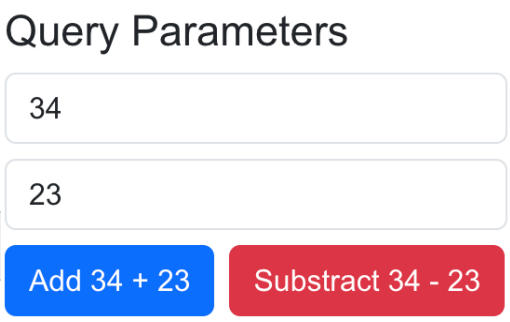


Meanwhile, in the React Web application, let's create a React component to test the new routes from our Web application. Web applications that interact with server applications are often referred to as ***client applications*** since they are the ***client*** in an application built using a ***client server architecture***. Create the component below that declares state variables ***a*** and ***b***, encodes the values in hyperlinks, and when you click them, server responds with the addition or subtraction of the parameters. Note that the name of the component is arbitrary. The fact that it is called the same as the routes in the server is a coincidence. Also helps us keep track of which UI components on the client are related to the server resources. On your own, create links that invoke the multiply and divide routes you implemented earlier. Import the new component in your ***Lab5*** component and confirm that clicking the links generates the expected response. Confirm all the routes work when you click the links in the browser.

app/labs/lab5/PathParameters.tsx ***C***

```
import React, { useState } from "react";
const HTTP_SERVER = process.env.NEXT_PUBLIC_HTTP_SERVER;
export default function PathParameters() {
  const [a, setA] = useState("34");
  const [b, setB] = useState("23");
  return (
    <div>
      <h3>Path Parameters</h3>
      <FormControl className="mb-2" id="wd-path-parameter-a" type="number" defaultValue={a}
             onChange={(e) => setA(e.target.value)}/>
      <FormControl className="mb-2" id="wd-path-parameter-b" type="number" defaultValue={b}
             onChange={(e) => setB(e.target.value)}/>
      <a className="btn btn-primary me-2" id="wd-path-parameter-add"
         href={`${HTTP_SERVER}/lab5/add/${a}/${b}`}>
         Add {a} + {b}
      </a>
      <a className="btn btn-danger" id="wd-path-parameter-subtract"
         href={`${HTTP_SERVER}/lab5/subtract/${a}/${b}`}>
         Substract {a} - {b}
      </a>
      <hr />
    </div>
  );
}
```

5.2.2.2 Sending Data to a Server with Query Parameters



React applications can also send data to servers by encoding it as a ***query string*** after the ***question mark*** character (?) at the end of a URL. A ***query string*** consists of a list of name value pairs separated by the ***ampersand*** character (&) as shown below.

http://localhost:4000/lab5/calculator?operation=add&a=2&b=4

In ***Lab5/QueryParameters.js***, in your server application, create a route that can parse an ***operation*** and its parameters ***a*** and ***b*** as shown below. If the ***operation*** is ***add*** the route responds with the addition of the parameters. If the ***operation*** is ***subtract***, the route responds with the subtraction of the parameters. On your own, also handle operations ***multiply*** and ***divide***. Import ***QueryParameters.js*** in ***Lab5/index.js*** and pass it a reference of ***app*** (not shown).

Lab5/QueryParameters.js ***S***

```
export default function QueryParameters(app) {
 const calculator = (req, res) => {
   const { a, b, operation } = req.query;                    // retrieve a, b, and operation parameters in query
   let result = 0;
   switch (operation) {
     case "add":
       result = parseInt(a) + parseInt(b);                   // parse as integers since parameters are strings
       break;
     case "subtract":
       result = parseInt(a) - parseInt(b);
       break;
     // implement multiply and divide on your own
     default:
       result = "Invalid operation";
   }
   res.send(result.toString());                              // convert to string otherwise browser interprets
 };                                                          // as a status code
 app.get("/lab5/calculator", calculator);                    // e.g., lab5/calculator?a=5&b=2&operation=add
}
```

In a new component ***QueryParameters.tsx***, in your React Web application, create hyperlinks to test the new route as shown below. You'll need to declare ***const removeServer*** initialized to the environment variable ***NEXT_PUBLIC_HTTP_SERVER***. Confirm the following hyperlinks work as expected.

Test Hyperlinks	*Confirm Response*
http://localhost:4000/lab5/calculator?operation=add&a=34&b=23	57
http://localhost:4000/lab5/calculator?operation=subtract&a=34&b=23	11

app/labs/lab5/QueryParameters.tsx ***C***

```
<div id="wd-query-parameters">
  <h3>Query Parameters</h3>
  <FormControl id="wd-query-parameter-a"
         className="mb-2"
         defaultValue={a} type="number"
         onChange={(e) => setA(e.target.value)} />
  <FormControl id="wd-query-parameter-b"
         className="mb-2"
         defaultValue={b} type="number"
```

```
        onChange={(e) => setB(e.target.value)} />
  <a id="wd-query-parameter-add"
     href={`${HTTP_SERVER}/lab5/calculator?operation=add&a=${a}&b=${b}`}>
    Add {a} + {b}
  </a>
  <a id="wd-query-parameter-subtract"
     href={`${HTTP_SERVER}/lab5/calculator?operation=subtract&a=${a}&b=${b}`}>
    Substract {a} - {b}
  </a>
  {/* create additional links to test multiply and divide. use IDs starting with wd-query-parameter- */}
  <hr />
</div>
```

5.2.2.3 On Your Own

On your own, remember to implement ***multiply*** and ***divide*** requests on the client and server that demonstrate multiplying and dividing numbers ***encoded in the request's path***. Now implement the same operations again, ***multiply*** and ***divide*** on the server and client that demonstrate, but multiplying and dividing parameters ***encoded in the query string***.

5.2.3 Working with Remote Objects on a Server

The examples so far have demonstrated working with integers and strings, but all primitive datatypes work as well, including objects and arrays. The example below declares an ***assignment*** object accessible at the route ***/lab5/assignment***. Import it to your ***Lab5/index.js*** in your server.

Lab5/WorkingWithObjects.js ***S***

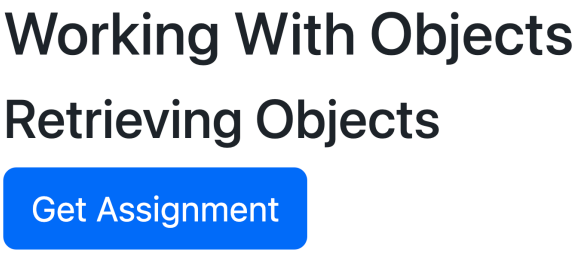
```
const assignment = {                                    // object state persists as long
  id: 1, title: "NodeJS Assignment",                    // as server is running
  description: "Create a NodeJS server with ExpressJS", // changes to the object persist
  due: "2021-10-10", completed: false, score: 0,        // rebooting server
};                                                      // resets the object
export default function WorkingWithObjects(app) {
  const getAssignment = (req, res) => {

    res.json(assignment);                               // use .json() instead of .send() if you know
  };                                                    // the response is formatted as JSON
  app.get("/lab5/assignment", getAssignment);
};
```

5.2.3.1 Retrieving Objects from a Server

In your React project, create a ***WorkingWithObjects*** component to test the new route as shown below. Import it in ***Lab5*** and confirm that http://localhost:4000/lab5/assignment responds with ***assignment*** object.

app/labs/lab5/WorkingWithObjects.tsx ***C***

```
import React, { useState } from "react";
const HTTP_SERVER = process.env.NEXT_PUBLIC_HTTP_SERVER;
export default function WorkingWithObjects() {
  return (
    <div id="wd-working-with-objects">
      <h3>Working With Objects</h3>
      <h4>Retrieving Objects</h4>
      <a id="wd-retrieve-assignments" className="btn btn-primary"
         href={`${HTTP_SERVER}/lab5/assignment`}>
        Get Assignment
      </a><hr/>
    </div>
);}
```

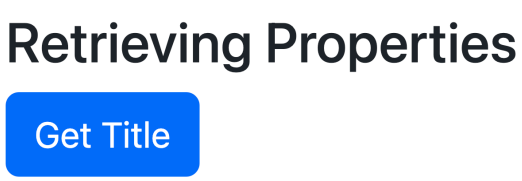


5.2.3.2 Retrieving Object Properties from a Server

We can retrieve individual properties in an object such as the ***title*** shown below.

Lab5/WorkingWithObjects.js ***S***

```
export default function WorkingWithObjects(app) {
  const getAssignment = (req, res) => {
    res.json(assignment);
  };
  const getAssignmentTitle = (req, res) => {
    res.json(assignment.title);
  };
  app.get("/lab5/assignment/title", getAssignmentTitle);
  app.get("/lab5/assignment", getAssignment);
}
```



Confirm that the http://localhost:4000/lab5/assignment/title hyperlink below retrieves the assignment's title. In ***WorkingWithObjects***, add a link that retrieves the title as shown below. Confirm that clicking the link retrieves the assignment's title.

app/labs/lab5/WorkingWithObjects.tsx ***C***

```
      ...
      <h4>Retrieving Objects</h4>
      <a id="wd-retrieve-assignments" className="btn btn-primary"
         href={`${HTTP_SERVER}/lab5/assignment`}>
        Get Assignment
      </a><hr/>
      <h4>Retrieving Properties</h4>
      <a id="wd-retrieve-assignment-title" className="btn btn-primary"
         href={`${HTTP_SERVER}/lab5/assignment/title`}>
        Get Title
      </a><hr/>
      ...
```

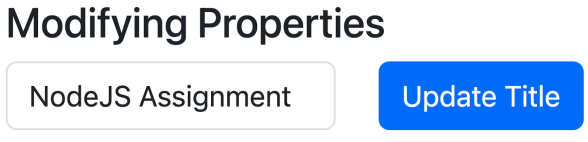
5.2.3.3 Modifying Objects in a Server

We can also use routes to modify objects or individual properties as shown below. The route retrieves the new title from the path and updates the ***assignment***'s object's ***title*** property.

Lab5/WorkingWithObjects.js ***S***

```
const setAssignmentTitle = (req, res) => {
  const { newTitle } = req.params;                    // changes to objects in the server
  assignment.title = newTitle;                        // persist as long as the server is running
  res.json(assignment);                               // rebooting the server resets the object state
};
app.get("/lab5/assignment/title/:newTitle", setAssignmentTitle);
```

In ***WorkingWithObjects*** component in your React project, create an ***assignment*** state variable to test editing the ***assignment*** object on the server. Create an input field where we can type the new assignment title, and a link that invokes the route that updates the title. Confirm that you can change the assignment's title.



app/labs/lab5/WorkingWithObjects.tsx ***C***

```
import React, { useState } from "react";
```

```
const HTTP_SERVER = process.env.NEXT_PUBLIC_HTTP_SERVER;
export default function WorkingWithObjects() {                    // create a state variable that holds
  const [assignment, setAssignment] = useState({                  // default values for the form below.
    id: 1, title: "NodeJS Assignment",                            // eventually we'll fetch this initial
    description: "Create a NodeJS server with ExpressJS",         // data from the server and populate
    due: "2021-10-10", completed: false, score: 0,                // the form with the remote data so
  });                                                             // we can modify it here in the UI
  const ASSIGNMENT_API_URL = `${HTTP_SERVER}/lab5/assignment`
  return (
    <div>
      <h3 id="wd-working-with-objects">Working With Objects</h3>
      <h4>Modifying Properties</h4>                               // encode the title in the URL that
      <a id="wd-update-assignment-title"                          // updates the title
         className="btn btn-primary float-end"
         href={`${ASSIGNMENT_API_URL}/title/${assignment.title}`}>
        Update Title </a>
      <FormControl className="w-75" id="wd-assignment-title"      // form element to edit local state variable
        defaultValue={assignment.title} onChange={(e) =>          // used to encode in URL that updates
          setAssignment({ ...assignment, title: e.target.value })}/>  // property in remote object
      <hr />
      ...
    </div> );}
```

5.2.3.4 On Your Own

Now, on your own, create a ***module*** object with ***string*** properties ***id***, ***name***, ***description***, and ***course***. Feel free to use values of your choice.

- Create a route that responds with the ***module*** object, mapped to ***/lab5/module***
- In the UI, create a link labeled ***Get Module*** that retrieves the ***module*** object from the server mapped at ***/lab5/module***
- Confirm that clicking the link retrieves the module.
- Create another route mapped to ***/lab5/module/name*** that retrieves the ***name*** of the ***module*** created earlier
- In the UI, create a hyperlink labeled ***Get Module Name*** that retrieves the ***name*** of the ***module*** object
- Confirm that clicking the link retrieves the module's name.

On your own, in ***WorkingWithObjects.tsx***, create a ***module*** state variable to test editing the ***module*** object on the server. Create an input field where we can type the new module name, and a link that invokes the route that updates the name. Confirm that you can change the module's name. Create routes and a corresponding UI that can modify the ***score*** and ***completed*** properties of the ***assignment*** object. In the React application, create an input field of type ***number*** where you can type the new ***score*** and an input field of type ***checkbox*** where you can select the ***completed*** property. Create a link that updates the ***score*** and another link that updates the ***completed*** property. For the module, create routes and UI to edit the module's description.

5.2.4 Working with Remote Arrays on a Server

Now let's work with something a little more challenging. Create an array of objects and explore how to retrieve, add, remove, and update the array. Working with a collection of objects requires a general set of operations often referred to as ***CRUD*** or ***create***, ***read***, ***update***, and ***delete***. These operations capture common interactions with any collection of data such as ***creating*** and adding new instances to the collection, ***reading*** or retrieving items in a collection, ***updating*** or modifying items in a collection, and ***deleting*** or removing items from a collection.

5.2.4.1 Retrieving Arrays from a Server

Let's first create the array data structure containing several todo objects. The most common integration between a client and server application is for a client application to retrieve all the instances of some collection. To illustrate this, create a route that retrieves the array of todo objects as shown below. Import the new route to ***Lab5/index.js***. Point the browser to http://localhost:4000/lab5/todos and confirm the server responds with the array of todos.

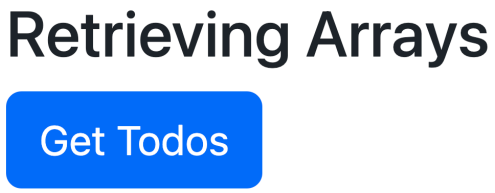
Lab5/WorkingWithArrays.js *S*

```
let todos = [
  { id: 1, title: "Task 1", completed: false },
  { id: 2, title: "Task 2", completed: true },
  { id: 3, title: "Task 3", completed: false },
  { id: 4, title: "Task 4", completed: true },
];
export default function WorkingWithArrays(app) {
  const getTodos = (req, res) => {
    res.json(todos);
  };
  app.get("/lab5/todos", getTodos);
};
```

In a new React component, create a hyperlink to test retrieving all todo objects in the array from the client. Add the new component to the ***Lab5*** complement and confirm that clicking the link retrieves the array.

app/Labs/Lab5/WorkingWithArrays.tsx *C*

```
const HTTP_SERVER = process.env.NEXT_PUBLIC_HTTP_SERVER;
export default function WorkingWithArrays() {
  const API = `${HTTP_SERVER}/lab5/todos`;
  return (
    <div id="wd-working-with-arrays">
      <h3>Working with Arrays</h3>
      <h4>Retrieving Arrays</h4>
      <a id="wd-retrieve-todos" className="btn btn-primary" href={API}>
        Get Todos </a><hr/>
    </div>
);}
```



5.2.4.2 Retrieving Data From a Server by Primary Key

Another common operation is to retrieve a particular item from an array by its primary key, e.g., its ID property. The convention is to encode the ID of the item of interest as a path parameter. Note that we could have chosen to encode the ID as query parameter instead, but it is best practice to encode identifiers in the path instead. The example below parses the ID as a path parameter, finds the corresponding item, and responds with the item.

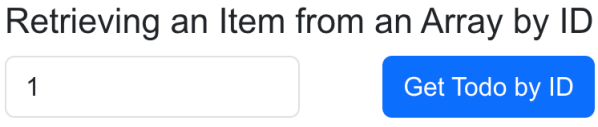
Lab5/WorkingWithArrays.js

```
  ...
  const getTodos = (req, res) => { ... }
  const getTodoById = (req, res) => {
    const { id } = req.params;
    const todo = todos.find((t) => t.id === parseInt(id));
    res.json(todo);
  };
  app.get("/lab5/todos", getTodos);
  app.get("/lab5/todos/:id", getTodoById);
  ...
```

Add a hyperlink to the React component to test retrieving an item from the array by its primary key. Confirm that you can type the ID in the UI and clicking the hyperlink retrieves the corresponding item.

app/Labs/Lab5/WorkingWithArrays.tsx

```
import React, { useState } from "react";
export default function WorkingWithArrays() {
      ...
      const [todo, setTodo] = useState({id: "1"});
      ...
```



```
      <h4>Retrieving Arrays</h4>
      <a id="wd-retrieve-todos" className="btn btn-primary" href={API}>
        Get Todos
      </a><hr/>
      <h4>Retrieving an Item from an Array by ID</h4>
      <a id="wd-retrieve-todo-by-id" className="btn btn-primary float-end" href={`${API}/${todo.id}`}>
        Get Todo by ID
      </a>
      <FormControl id="wd-todo-id" defaultValue={todo.id} className="w-50"
        onChange={(e) => setTodo({ ...todo, id: e.target.value })} />
      <hr />
      ...
```

5.2.4.3 Filtering Data From a Server With a Query String

The convention for retrieving a particular item from a collection is to encode the item's ID as a path parameter, e.g., ***/todos/123***. Another convention is that if the primary key is not provided, then the interpretation is that we want the entire collection of items, e.g., ***/todos***. We can also want to retrieve items by some other criteria other than the item's ID such as the item's ***title*** or ***completed*** properties. The best practice in this case is to use query strings instead of path parameters when filtering items by properties other than the primary key, e.g., ***/todos?completed=true***. The example below refactors the ***/lab5/todos*** to handle the case when we want to filter the array by the ***completed*** query parameter.

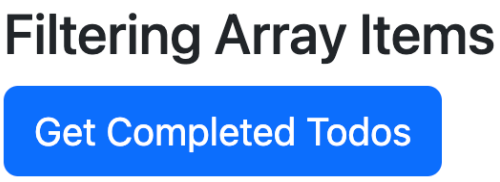
Lab5/WorkingWithArrays.js

```
const todos = [  { id: 1, title: "Task 1", completed: false },  { id: 2, title: "Task 2", completed: true },
                 { id: 3, title: "Task 3", completed: false },  { id: 4, title: "Task 4", completed: true },  ];
export default function WorkingWithArrays(app) {
  const getTodos = (req, res) => {
    const { completed } = req.query;
    if (completed !== undefined) {
      const completedBool = completed === "true";
      const completedTodos = todos.filter((t) => t.completed === completedBool);
      res.json(completedTodos);
      return;
    }
    res.json(todos);
  };
  const getTodoById = (req, res) => { ... }
  app.get("/lab5/todos", getTodos);
  app.get("/lab5/todos/:id", getTodoById);
};
```

Add a hyperlink to the React component to test retrieving all completed todos.

app/labs/lab5/WorkingWithArrays.tsx

```
  ...
  <h3>Retrieving Arrays</h3>
  ...
  <h3>Filtering Array Items</h3>
  <a id="wd-retrieve-completed-todos" className="btn btn-primary"
     href={`${API}?completed=true`}>
    Get Completed Todos
  </a><hr/>
  ...
```



5.2.4.4 Creating New Data in a Server

The examples we've seen so far have illustrated various ***read*** operations in our exploration of possible ***CRUD*** operations. Let's now take a look at the ***create*** operation. The example below demonstrates a route that creates a new item in the array and responds with the array now containing the new item. Note that it is implemented before the ***/lab5/todos/:id*** route, otherwise the ***:id*** path parameter would interpret the "***create***" in ***/lab5/todos/create*** as an ID, which would certainly

create an error trying to parse it as an integer. Also note that the ***newTodo*** creates default values including a unique identifier field ***id*** based on a timestamp. Eventually primary keys will be handled by a database later in the course. Finally note that the response consists of the entire ***todos*** array, which is convenient for us for now, but a more common implementation would be to respond with ***newTodo***.

Lab5/WorkingWithArrays.js

```
export default function WorkingWithArrays(app) {
  ...
  const createNewTodo = (req, res) => {
    const newTodo = {
      id: new Date().getTime(),
      title: "New Task",
      completed: false,
    };
    todos.push(newTodo);
    res.json(todos);
  };
  const getTodoById = (req, res) => { ... }
  app.get("/lab5/todos/create", createNewTodo);          // make sure to implement this BEFORE the /lab5/todos/:id
  app.get("/lab5/todos/:id", getTodoById);               // make sure to implement this route AFTER the
  ...                                                    // /lab5/todos/create route implemented above
```

Add a ***Create Todo*** hyperlink to the ***WorkingWithArrays*** component to test the new route. Confirm that clicking the link creates the new item in the array.

app/labs/lab5/WorkingWithArrays.tsx

```
  ...
  <h3>Creating new Items in an Array</h3>
  <a id="wd-retrieve-completed-todos" className="btn btn-primary"
     href={`${API}/create`}>
    Create Todo
  </a><hr/>
  ...
```

Creating new Items in an Array

Create Todo

5.2.4.5 Deleting Data from a Server

Next let's consider the ***delete*** operation in the ***CRUD*** family of operations. The convention is to encode the ID of the item to delete as a path parameter as shown below. We search for the item in the set of items and remove it. Typically we would respond with a status of success or failure, but for now we're responding with all the todos for now.

Lab5/WorkingWithArrays.js

```
  ...
  const getTodos = (req, res) => { ... }
  const createNewTodo = (req, res) => { ... }
  const getTodoById = (req, res) => { ... }
  const removeTodo = (req, res) => {
    const { id } = req.params;
    const todoIndex = todos.findIndex((t) => t.id === parseInt(id));
    todos.splice(todoIndex, 1);
    res.json(todos);
  };
  app.get("/lab5/todos/:id/delete", removeTodo);
  app.get("/lab5/todos", getTodos);
  app.get("/lab5/todos/create", createNewTodo);
  app.get("/lab5/todos/:id", getTodoById);
  ...
```

Deleting from an Array

2

Delete Todo with ID = 2

To test the new route, create a link that encodes the todo's ID in a hyperlink to delete the corresponding item. We'll use the ***todo*** state variable created earlier to type the ID of the item we want to remove. Confirm that you can type the ID of an item, click the hyperlink and that the corresponding item is removed from the array.

app/labs/lab5/WorkingWithArrays.tsx

```
...
<h3>Removing from an Array</h3>
<a id="wd-remove-todo" className="btn btn-primary float-end" href={`${API}/${todo.id}/delete`}>
   Remove Todo with ID = {todo.id} </a>
<FormControl defaultValue={todo.id} className="w-50" onChange={(e) => setTodo({ ...todo, id: e.target.value })}/><hr/>
...
```

5.2.4.6 Updating Data on a Server

Finally let's consider the ***update*** operation in the ***CRUD*** family of operations. The convention is to encode the ID of the item to update as a path parameter as shown below. We search for the item in the set of items and update it. Typically we would respond with a status of success or failure, but for now we're responding with all the todos. Group the callback functions together towards the top of the routing file and the route declarations grouped towards the bottom of the file.

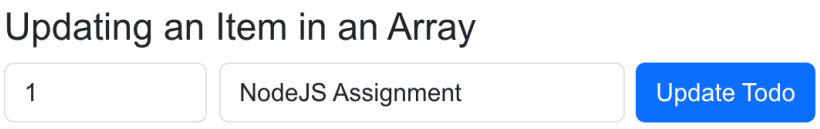
Lab5/WorkingWithArrays.js

```
  ...
  const updateTodoTitle = (req, res) => {
    const { id, title } = req.params;
    const todo = todos.find((t) => t.id === parseInt(id));
    todo.title = title;
    res.json(todos);
  };
  app.get("/lab5/todos/:id/title/:title", updateTodoTitle);
 ...
```

To test the new route, add an input field to edit the ***title*** property and a link that encodes both the ID of the item and the new value of the ***title*** property as shown below

app/labs/lab5/WorkingWithArrays.tsx

```
const HTTP_SERVER = process.env.NEXT_PUBLIC_HTTP_SERVER;
export default function WorkingWithArrays() {
  const [todo, setTodo] = useState({
    id: "1",
    title: "NodeJS Assignment",
    description: "Create a NodeJS server with ExpressJS",
    due: "2021-09-09",
    completed: false,
  });
  return (
    <div>
      <h2>Working with Arrays</h2>
      ...
      <h3>Updating an Item in an Array</h3>
      <a href={`${API}/${todo.id}/title/${todo.title}`} className="btn btn-primary float-end">
        Update Todo</a>
      <FormControl defaultValue={todo.id} className="w-25 float-start me-2"
        onChange={(e) => setTodo({ ...todo, id: e.target.value })}/>
      <FormControl defaultValue={todo.title} className="w-50 float-start"
            onChange={(e) => setTodo({ ...todo, title: e.target.value }) }/>
      <br /><br /><hr />
      ...
```



5.2.4.7 On Your Own

Using the exercises so far as examples, implement routes and corresponding UI that allows editing a ***completed*** and ***description*** properties of ***todo*** items identified by their ID. Create the routes below in ***Lab5/index.js*** in the Node.js HTTP server project. In ***WorkingWithArrays*** component, add a text input field to edit the ***description*** and a checkbox input field to edit the ***completed*** property. Create a link that updates the ***description*** of the todo item whose ***id*** is encoded in the URL and another link that updates the ***completed*** property of the todo item whose ***id*** is encoded in the URL.

Property	*Routes*	*Response*	*Test*
completed	***/lab5/todos/:id/completed/:completed***	todos	Complete Todo ID = 1
description	***/lab5/todos/:id/description/:description***	todos	Describe Todo ID = 1

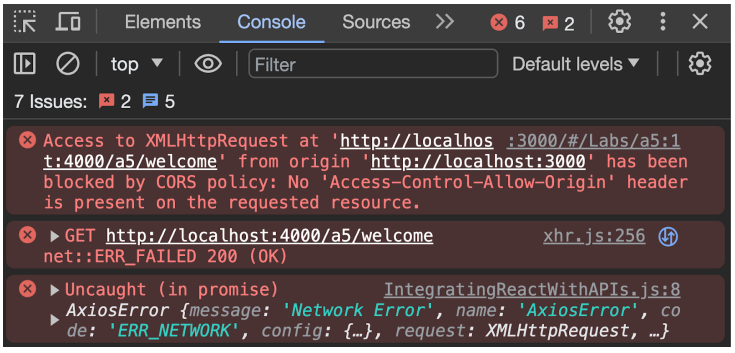
5.2.5 Asynchronous Communication with HTTP Servers

The exercises explored so far sent data encoded in the URL of hyperlinks. The links navigated to a separate browser window that displayed the server response. Even though we were able to send data to the server and affect changes to the server data, we have not considered how those changes can affect the user interface. Let's explore how to fully integrate the user interface with the server by sending and receiving HTTP requests and responses asynchronously.

5.2.5.1 Asynchronous JavaScript and XML

JavaScript Web applications, such as React applications, can communicate with server applications using a technology called ***AJAX*** or ***Asynchronous JavaScript and XML***. Using ***AJAX*** JavaScript applications can send and retrieve HTTP requests and response asynchronously from client JavaScript applications to remote HTTP server. Although ***XML*** was the original data format in ***AJAX***, ***JSON*** has overtaken as the dominant data format in modern Web applications, but the ***AJAX*** label still applies nevertheless. ***Axios*** is a popular JavaScript library that React user interface applications can use to communicate with servers using ***AJAX***. Install the library at the root of the React Web application project as shown below.

```
$ npm install axios
```

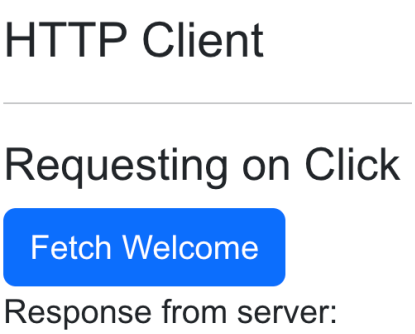


Let's use the same server routes implemented in earlier exercises, but instead of clicking on hyperlinks in the React client, we'll use ***axios*** to programmatically invoke the URLs giving us a chance to capture and handle the responses from the server and render the response in the user interface. The code below illustrates how to use the ***axios*** library to send an asynchronous request to the server and then capture the response in the user interface, without navigating to the URL, away from the current window. The ***fetchWelcomeOnClick*** function is tagged as ***async*** since it uses ***axios.get()*** to asynchronously send a request to the server, and returns the response from the server. Create the component below and import it in the ***Lab5*** component. Open the ***Web Dev Tools*** and confirm that clicking the ***Fetch Welcome*** button causes the ***CORS*** error shown here on the right.

app/labs/lab5/HttpClient.tsx

```
import React, { useEffect, useState } from "react";
```

```
import axios from "axios";

const HTTP_SERVER = process.env.NEXT_PUBLIC_HTTP_SERVER;

export default function HttpClient() {
  const [welcomeOnClick, setWelcomeOnClick] = useState("");

  const fetchWelcomeOnClick = async () => {
    const response = await axios.get(`${HTTP_SERVER}/lab5/welcome`);
    setWelcomeOnClick(response.data);
  };
  return (
    <div>
      <h3>HTTP Client</h3> <hr />
      <h4>Requesting on Click</h4>
      <button className="btn btn-primary me-2" onClick={fetchWelcomeOnClick}>
        Fetch Welcome
      </button> <br />
      Response from server: <b>{welcomeOnClick}</b>
    </div>
  );
}
```



5.2.5.2 Configuring Cross Origin Request Sharing (CORS)

Servers and browsers limit ***JavaScript*** programs to only be able to communicate with the servers from where they are downloaded from. Since our ***React*** application is running locally from ***localhost:5173***, then they would only be able to communicate back to a server running on ***localhost:5173***, but our server is running on ***localhost:4000***, so when our ***JavaScript*** components try to communicate with ***localhost:4000***, the browser considers a different domain as a security risk, stops the communication and throws a ***CORS*** exception. ***CORS*** stands for ***Cross Origin Request Sharing***, which governs the policies and mechanisms of how various resources can be shared across different domains or ***origins***. Browsers enforce ***CORS*** policies by first checking with the server if they are ok with receiving requests from different domains. If the server responds affirmatively, then browsers let the requests go through, otherwise they'll consider the attempt as a violation of ***CORS*** security policy, abort the request, and throw the exception. We can configure the ***CORS*** security policies by installing the ***cors*** Node.js library as shown below.

```
$ npm  install  cors
```

In ***index.js***, import the ***cors*** library and configure it as shown below to allow all requests from any origin. We'll narrow down this policy in a later chapter. Restart the server and refresh the React application. Confirm that the user interface is able to retrieve the ***Welcome to Lab 5*** message from the server without errors.

index.js

```
import express from "express";
import Lab5 from "./Lab5/index.js";
import cors from "cors";
const app = express();
app.use(cors());                    // make sure cors is used right after creating the app
Lab5(app);                          // express instance
app.listen(4000);
```

5.2.5.3 Creating a Client Library

The current ***HttpClient.tsx*** implementation makes a request to the server from the component itself using the ***axios*** library. In addition to ***retrieving*** (or ***reading***) data from the server, there will be other ***CRUD*** operations to ***create***, ***update***, and ***delete*** needed to interact with the server. Instead of implementing these in a React component, it's better to implement these in a reusable ***client library*** that can be shared across several user interface components. Move the ***axios.get()*** in ***HttpClient.tsx*** to a separate file called ***client.ts*** as shown below.

app/labs/lab5/client.ts

```
import axios from "axios";
const HTTP_SERVER = process.env.NEXT_PUBLIC_HTTP_SERVER;
export const fetchWelcomeMessage = async () => {
  const response = await axios.get(`${HTTP_SERVER}/lab5/welcome`);
  return response.data;
};
```

Now refactor ***HelloClient.ts*** to use the ***client.ts*** as shown below. Confirm that clicking on ***Fetch Welcome*** still works.

app/labs/lab5/HttpClient.tsx

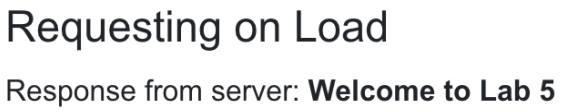
```
import * as client from "./client";
export default function HttpClient() {
  const [welcomeOnClick, setWelcomeOnClick] = useState("");
  const fetchWelcomeOnClick = async () => {
    const message = await client.fetchWelcomeMessage();
    setWelcomeOnClick(message);
  };
  return ( ... );
}
```

5.2.5.4 Retrieving Data from a Server on Component Load

The previous exercise fetched data from the server when the user requested it. Often times we need to retrieve data from the server when you first navigate to a screen or a component is first loaded and displayed. Use React's ***useEffect*** hook function as shown below to invoke the ***fetchWelcomeOnLoad*** when a component or screen first loads. Now, when the ***HttpClient*** loads, the ***useEffect*** invokes ***fetchWelcomeOnLoad*** which retrieves the message from the server and sets the new ***welcomeOnLoad*** state variable. Confirm that if you refresh the screen, the ***welcome*** message appears without having to click on the ***Fetch Welcome*** button.

app/labs/lab5/HttpClient.tsx

```
import React, { useEffect, useState } from "react";
import * as client from "./client";
export default function HttpClient() {
  const [welcomeOnClick, setWelcomeOnClick] = useState("");
  const [welcomeOnLoad, setWelcomeOnLoad] = useState("");
  const fetchWelcomeOnClick = async () => {
    const message = await client.fetchWelcomeMessage();
    setWelcomeOnClick(message);
  };
  const fetchWelcomeOnLoad = async () => {
    const welcome = await client.fetchWelcomeMessage();
    setWelcomeOnLoad(welcome);
  };
  useEffect(() => {
    fetchWelcomeOnLoad();
  }, []);
  return (
    <div>
      <h3>HTTP Client</h3> <hr />
      <h4>Requesting on Click</h4>
      ...
      <hr />
      <h4>Requesting on Load</h4>
      Response from server: <b>{welcomeOnLoad}</b>
      <hr />
    </div>
);}
```



5.2.5.5 Working with Remote Objects on a Server Asynchronously

Let's now revisit the APIs that worked with the ***assignment*** object in ***WorkingWithObjects*** and create an asynchronous version. Let's add client functions to ***client.ts*** to fetch the assignment object from the server and update its title as shown below.

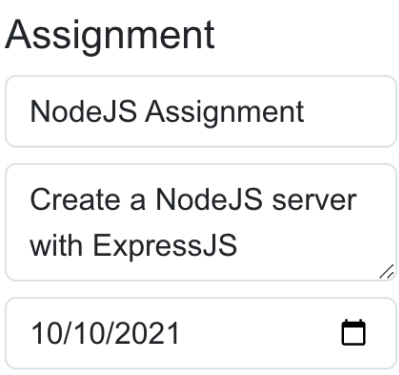
app/labs/lab5/client.ts

```
import axios from "axios";
const HTTP_SERVER = process.env.NEXT_PUBLIC_HTTP_SERVER;
export const fetchWelcomeMessage = async () => {
  const response = await axios.get(`${HTTP_SERVER}/lab5/welcome`);
  return response.data;
};
const ASSIGNMENT_API = `${HTTP_SERVER}/lab5/assignment`;
export const fetchAssignment = async () => {
  const response = await axios.get(`${ASSIGNMENT_API}`);
  return response.data;
};
export const updateTitle = async (title: string) => {
  const response = await axios.get(`${ASSIGNMENT_API}/title/${title}`);
  return response.data;
};
```

Then, in a new ***WorkingWithObjectsAsynchronously*** component, create a UI that fetches the assignment on load and then allows you to edit its title. Import the component in ***Lab5*** and confirm that the assignment is displayed on load.

app/labs/lab5/WorkingWithObjectsAsynchronously.tsx

```
import React, { useEffect, useState } from "react";
import * as client from "./client";
export default function WorkingWithObjectsAsynchronously() {
  const [assignment, setAssignment] = useState<any>({});
  const fetchAssignment = async () => {
    const assignment = await client.fetchAssignment();
    setAssignment(assignment);
  };
  useEffect(() => {
    fetchAssignment();
  }, []);
  return (
    <div id="wd-asynchronous-objects">
      <h3>Working with Objects Asynchronously</h3>
      <h4>Assignment</h4>
      <FormControl defaultValue={assignment.title} className="mb-2"
        onChange={(e) => setAssignment({ ...assignment, title: e.target.value }) } />
      <FormControl rows={3} defaultValue={assignment.description} className="mb-2"
        onChange={(e) => setAssignment({ ...assignment, description: e.target.value }) }/>
      <FormControl type="date" className="mb-2" defaultValue={assignment.due}
        onChange={(e) => setAssignment({ ...assignment, due: e.target.value })} />
      <div className="form-check form-switch">
        <input className="form-check-input" type="checkbox" id="wd-completed"
               defaultChecked={assignment.completed}
          onChange={(e) => setAssignment({ ...assignment, completed: e.target.checked }) } />
        <label className="form-check-label" htmlFor="wd-completed"> Completed </label>
      </div>
      <pre>{JSON.stringify(assignment, null, 2)}</pre>
      <hr />
    </div>
);}
```



Now let's add a button to update the assignment's title. Change the assignment's title, refresh the screen and confirm that the title has changed.

app/labs/lab5/WorkingWithObjectsAsynchronously.tsx

```
export default function WorkingWithObjectsAsynchronously() {
  const [assignment, setAssignment] = useState<any>({});
  const fetchAssignment = async () => {
    const assignment = await client.fetchAssignment();
    setAssignment(assignment);
  };
  const updateTitle = async (title: string) => {
    const updatedAssignment = await client.updateTitle(title);
    setAssignment(updatedAssignment);
  };
  ...
  return (
    <div id="wd-asynchronous-objects">
      <h3>Working with Objects Asynchronously</h3>
      <h4>Assignment</h4>
      ...
      <button className="btn btn-primary me-2" onClick={() => updateTitle(assignment.title)} >
        Update Title
      </button>
      <pre>{JSON.stringify(assignment, null, 2)}</pre>
      <hr />
    </div>
);}
```

5.2.5.6 Working with Remote Arrays on a Server Asynchronously

Now let's do the same thing to the arrays. Let's use ***axios*** so that we can manipulate remote arrays on the server from the user interface and update a user interface to reflect the changes in the remote array. We'll implement several client functions in ***client.ts*** that use ***axios*** to communicate with the server and we'll use them from a new component that will render the remote array in the use interface.

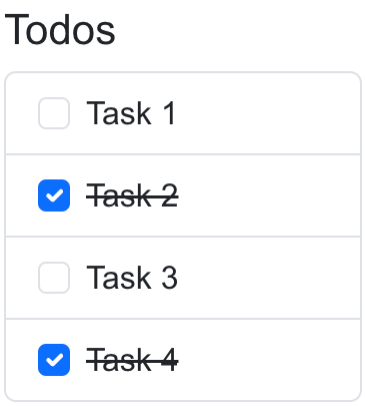
app/labs/lab5/client.ts

```
const TODOS_API = `${HTTP_SERVER}/lab5/todos`;
export const fetchTodos = async () => {
  const response = await axios.get(TODOS_API);
  return response.data;
};
```

The exercise below fetches the ***todos*** from the server and populate ***todos*** state variable which we can then render as a list of todos when the component loads. Confirm that the todos render when the component first loads.

app/labs/lab5/WorkingWithArraysAsynchronously.tsx

```
import React, { useState, useEffect } from "react";
import * as client from "./client";
export default function WorkingWithArraysAsynchronously() {
  const [todos, setTodos] = useState<any[]>([]);
  const fetchTodos = async () => {
    const todos = await client.fetchTodos();
    setTodos(todos);
  };
  useEffect(() => {
    fetchTodos();
  }, []);
  return (
    <div id="wd-asynchronous-arrays">
      <h3>Working with Arrays Asynchronously</h3>
      <h4>Todos</h4>
      <ListGroup>
        {todos.map((todo) => (
          <ListGroupItem key={todo.id}>
            <input type="checkbox" className="form-check-input me-2"
                   defaultChecked={todo.completed}/>
            <span style={{ textDecoration: todo.completed ? "line-through" : "none" }}>
```



```
            {todo.title} </span>
          </ListGroupItem>
        ))}
      </ListGroup> <hr />
    </div>
);}
```

5.2.5.7 Deleting Data from a Server Asynchronously

In ***client.ts***, add a ***removeTodo*** client function that sends a ***delete*** request to the server. The server will respond with an array with the surviving todos.

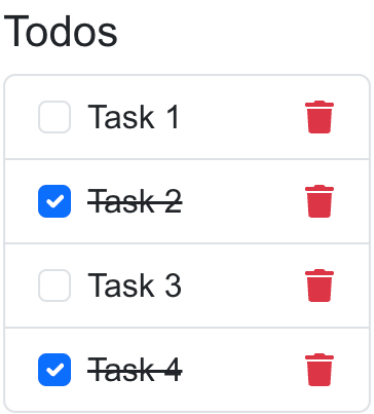
app/labs/lab5/client.ts

```
export const removeTodo = async (todo: any) => {
  const response = await axios.get(`${TODOS_API}/${todo.id}/delete`);
  return response.data;
};
```

In the ***WorkingWithArraysAsynchronously*** component, add ***remove*** buttons to each of the todos that invoke a new ***removeTodo*** function that uses the client to send asynchronous delete request to the server and updates the ***todos*** state variable with the surviving todos. Use a ***trashcan*** icon to represent the ***remove*** button as shown below. Confirm that clicking on the new ***remove*** buttons actually removes the corresponding todo.

app/labs/lab5/WorkingWithArraysAsynchronously.tsx

```
export default function WorkingWithArraysAsynchronously() {
  const [todos, setTodos] = useState<any[]>([]);
  const fetchTodos = async () => { ... };
  const removeTodo = async (todo: any) => {
    const updatedTodos = await client.removeTodo(todo);
    setTodos(updatedTodos);
  };
  ...
  return (
    <div id="wd-asynchronous-arrays">
      <h3>Working with Arrays Asynchronously</h3>
      <h4>Todos</h4>
      <ListGroup>
        {todos.map((todo) => (
          <ListGroupItem key={todo.id}>
            <FaTrash onClick={() => removeTodo(todo)}
                     className="text-danger float-end mt-1" id="wd-remove-todo"/>
            ...
          </ListGroupItem>
        ))}
      </ListGroup><hr />
    </div>
);}
```



5.2.5.8 Creating New Data in a Server Asynchronously

A previous exercise implemented server route to create new todo items. In the React's project ***client.ts*** implement a ***createNewTodo*** client function that requests creating a new todo item from the server as shown below.

app/labs/lab5/client.ts

```
export const createNewTodo = async () => {
  const response = await axios.get(`${TODOS_API}/create`);
  return response.data;
};
```

In the ***WorkingWithArraysAsynchronously*** component, add a ***+ button icon*** to invoke the ***createNewTodo*** client function and update the ***todos*** state variables with the ***todos*** from the server. Confirm that clicking the ***+ button icon*** actually creates a new todo.

app/labs/lab5/WorkingWithArraysAsynchronously.tsx

```
import { FaPlusCircle } from "react-icons/fa";
import * as client from "./client";
export default function WorkingWithArraysAsynchronously() {
  const [todos, setTodos] = useState<any[]>([]);
  const createNewTodo = async () => {
    const todos = await client.createNewTodo();
    setTodos(todos);
  };
  ...
  return (
    <div id="wd-asynchronous-arrays">
      <h3>Working with Arrays Asynchronously</h3>
      <h4> Todos <FaPlusCircle onClick={createNewTodo} className="text-success float-end fs-3" /> </h4>
      ...
    </div>
);}
```

5.2.6 Passing JSON Data to a Server in an HTTP Body

The exercises so far have sent data to the server as path and query parameters. This approach is limited to the maximum length of the URL string, and only string data types. Another concern is that data in the URL is sent over a network in clear text, so anyone snooping around between the client and server can see the data as plain text which is not a good option for exchanging sensitive information such as passwords and other personal data. A better approach is to encode the data as ***JSON*** in the ***HTTP request body*** which allows for arbitrarily large amounts of data as well as secure data encryption. To enable the server to parse ***JSON*** data from the ***request body***, add the following ***app.use()*** statement in ***index.js***. Make sure that it's implemented right after the ***CORS*** configuration statement. Now ***JSON*** data coming from the client is available in the ***request body*** in the ***request.body*** property in the server routes.

index.js

```
import Lab5 from "./Lab5/index.js";
import cors from "cors";
const app = express();
app.use(cors());
app.use(express.json());                                    // make sure this statement occurs AFTER
Lab5(app);                                                  // setting up CORS but BEFORE all the routes
app.listen(4000);
```

The hyperlinks and ***axios.get()*** in the exercises so far have sent data to the server using the ***HTTP GET method*** or ***verb***. HTTP defines several other ***HTTP methods*** or ***verbs*** including:

- ***GET*** - for retrieving data, but we've been also misusing it for creating, modifying and deleting data on the server. We'll start using it properly only for retrieving data
- ***POST*** - for creating new data typically embedded in the HTTP body
- ***PUT*** - for modifying existing data where updates are typically embedded in the HTTP body
- ***DELETE*** - for removing existing data
- ***OPTIONS*** - for retrieving allowed operations. Used to figure out if CORS policy allows communication with the other methods such as GET, POST, PUT and DELETE

The ***GET*** method, as the name suggests, is meant for only getting data from the server. We've been misusing it to implement routes that also creates, updates, and deletes data on the server. We did this mostly for academic purposes since it's the easiest HTTP method to work with. From now on we'll use the proper HTTP method for the right purpose.

5.2.6.1 Posting Data to Servers with HTTP POST Requests

To illustrate using the HTTP POST method, let's re-implement the route that creates new todos as shown below. The ***HTTP POST method*** takes the role of the verb meaning ***create***. Add the new ***app.post*** implementation highlighted in green below. Don't remove the old version highlighted in yellow so we don't break the other lab exercises. Note how the new implementation grabs the posted ***JSON*** data from ***req.body*** and uses it to define ***newTodo***. Also note that this version does not respond with the entire ***todos*** array and instead only responds with the newly created todo object instance. This is more reasonable since arrays can potentially be large and it would be expensive to transfer such large data structures over a network, especially if the client UI already has most of this data already displayed.

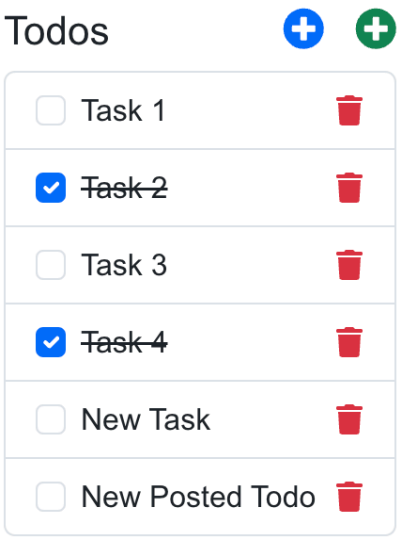
Lab5/WorkingWithArrays.js

```
  ...
  const createNewTodo = (req, res) => {...}
  const postNewTodo = (req, res) => {
    const newTodo = { ...req.body, id: new Date().getTime() };
    todos.push(newTodo);
    res.json(newTodo);
  };
  ...
  app.get("/lab5/todos/create", createNewTodo);
  app.post("/lab5/todos", postNewTodo);
  ...
```

Back in the user interface, add a new ***postNewTodo*** client function in ***client.ts*** that posts new ***todo*** objects to the server as shown below. Note the second argument in the ***axios.post()*** method containing the new ***todo*** object instance sent to the server. The response this time contains the ***todo*** instance added to the ***todos*** array in the server instead of all the todos on the server.

app/labs/lab5/client.ts

```
export const createNewTodo = async () => {
  const response = await axios.get(`${TODOS_API}/create`);
  return response.data;
};
export const postNewTodo = async (todo: any) => {
  const response = await axios.post(`${TODOS_API}`, todo);
  return response.data;
};
```



In the ***WorkingWithArraysAsynchronously*** component, create a new ***+ button icon*** to invoke the new ***postNewTodo*** client function to send a new ***todo*** object to the server containing a default ***title*** and ***completed*** properties. Append the new ***todo*** object created on the server, to the local ***todos*** state variable to update the user interface with the new todo as shown below. Color the new ***+ button icon*** a different color so it's distinguishable from the ***createNewTodo*** button. Confirm that you can add new items to the array when you click on the new ***+ button icon***.

app/labs/lab5/WorkingWithArraysAsynchronously.tsx

```
import { FaPlusCircle } from "react-icons/fa";
import * as client from "./client";
export default function WorkingWithArraysAsynchronously() {
  const [todos, setTodos] = useState<any[]>([]);
  const createNewTodo = async () => {
    const todos = await client.createNewTodo();
    setTodos(todos);
  };
  const postNewTodo = async () => {
    const newTodo = await client.postNewTodo({ title: "New Posted Todo", completed: false, });
    setTodos([...todos, newTodo]);
```

```
  };
  return (
    <div id="wd-asynchronous-arrays">
      <h3>Working with Arrays Asynchronously</h3>
      <h4>
        Todos
        <FaPlusCircle onClick={createNewTodo} className="text-success float-end fs-3"      id="wd-create-todo" />
        <FaPlusCircle onClick={postNewTodo}   className="text-primary float-end fs-3 me-3" id="wd-post-todo"   />
      </h4>
    </div>
);}
```

5.2.6.2 Deleting Data from Servers with HTTP DELETE Requests

Now that we have ***axios*** we can implement a better version of the remove operation. The current ***removeTodo*** implementation uses the ***HTTP GET*** method to request the server to remove data. The ***HTTP DELETE*** method is specifically suited for removing data from remote servers. In the server project, implement a better version of the delete operation as shown below. The new implementation uses the ***HTTP DELETE*** method declared in ***app.delete()*** which is distinct from ***app.get()*** for which we don't need the trailing ***/delete*** at the end of the ***URL***. Removing the element from the array is the same either way. Although we could again respond with the entire array of surviving todos, it is better to just respond with a success status and let the user interface update its state variable. This reduces unnecessary data communication between the client and server.

Lab5/WorkingWithArrays.js

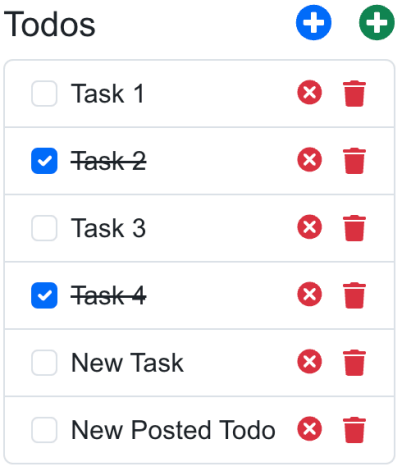
```
  ...
  const removeTodo = (req, res) => {
  const deleteTodo = (req, res) => {
    const { id } = req.params;
    const todoIndex = todos.findIndex((t) => t.id === parseInt(id));
    todos.splice(todoIndex, 1);
    res.sendStatus(200);
  };
  app.delete("/lab5/todos/:id", deleteTodo);
  app.get("/lab5/todos/:id/delete", removeTodo);
  ...
```

In the React project create a new client function called ***deleteTodo*** as shown below. Note how it's implemented using ***axios.delete*** instead of ***axios.get*** so that it matches the server's ***app.delete*** as well as the ***URL*** format without the trailing ***/delete***.

app/labs/lab5/client.ts

```
export const removeTodo = async (todo: any) => {
  const response = await axios.get(`${TODOS_API}/${todo.id}/delete`);
  return response.data;
};
export const deleteTodo = async (todo: any) => {
  const response = await axios.delete(`${TODOS_API}/${todo.id}`);
  return response.data;
};
```

In the user interface component ***WorkingWithArraysAsynchronously***, add another ***delete*** button to try this new ***deleteTodo*** client function, but use a different icon, say an ***X*** so as to not confuse it with the ***trash***. Note that the new implementation ignores the response from the server and instead filters the removed todo from the local state variable. This is fine for now, but the operation is too optimistic assuming the server successfully deleted the item from the array and updating the user interface without confirmation. Later we'll deal with errors from the server to make sure the local state variable in the user interface is in synch with the remote array on the server.



app/labs/lab5/WorkingWithArraysAsynchronously.tsx

```
import { TiDelete } from "react-icons/ti";
import * as client from "./client";
export default function WorkingWithArraysAsynchronously() {
  const [todos, setTodos] = useState<any[]>([]);
  ...
  const deleteTodo = async (todo: any) => {
    await client.deleteTodo(todo);
    const newTodos = todos.filter((t) => t.id !== todo.id);
    setTodos(newTodos);
  };
  ...
  return (
    ...
    <ListGroupItem key={todo.id}>
      <FaTrash onClick={() => removeTodo(todo)}  className="text-danger float-end mt-1"      id="wd-remove-todo" />
      <TiDelete onClick={() => deleteTodo(todo)} className="text-danger float-end me-2 fs-3" id="wd-delete-todo" />
    </ListGroupItem>
    ...
);}
```

5.2.6.3 Updating Data on Servers with HTTP PUT Requests

Use ***HTTP PUT*** to reimplement the route that updates an item in an array as shown below. The route replaces the todo item whose ID matches the ***id*** path parameter with a combination of the original todo object and properties in the ***req.body***. This overrides any properties in the original todo object with matching properties in the ***req.body***. Note that this new implementation does not respond with the ***todos*** array, but instead responds with a simple ***OK*** status code of ***200***. This is more reasonable since there is no need to respond with an entire array since the user interface already has the array cached in the browser and it can just update the item in the ***todos*** state variable.

Lab5/WorkingWithArrays.js

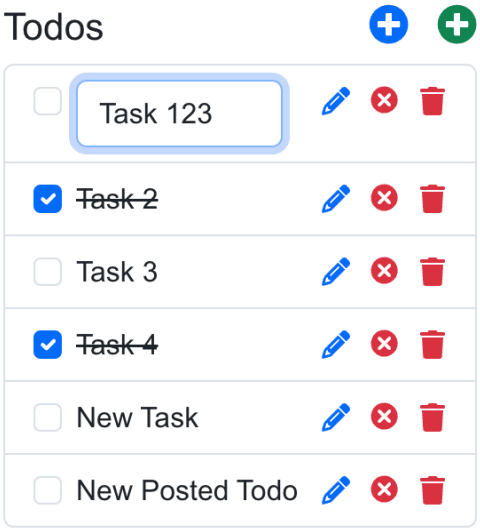
```
  ...
  const updateTodo = (req, res) => {
    const { id } = req.params;
    todos = todos.map((t) => {
      if (t.id === parseInt(id)) {
        return { ...t, ...req.body };
      }
      return t;
    });
    res.sendStatus(200);
  };
  app.put("/lab5/todos/:id", updateTodo);
  ...
```

Back in the user interface, in ***clients.ts*** add a new ***updateTodo*** function that ***puts*** updates to the server as shown below. Note the second argument in the ***axios.put()*** method containing the updated ***todo*** object instance sent to the server. The response contains a status.

app/labs/lab5/client.ts

```
export const updateTodo = async (todo: any) => {
  const response = await axios.put(`${TODOS_API}/${todo.id}`, todo);
  return response.data;
};
```

In the ***WorkingWithArraysAsynchronously*** component, add an input field that shows up when you click a new ***pencil icon*** by setting the ***todo***'s ***editing*** property to true. Pressing the ***Enter*** key sets the ***todo***'s ***editing*** property to false, and shows the updated ***title*** again. Add an ***onChange*** attribute to the ***completed*** checkbox so that it updates the corresponding property of the ***todo*** object. Confirm you can edit the ***title*** and ***completed*** properties of the ***todos***, and that the changes persist after refreshing the page.



app/labs/lab5/WorkingWithArraysAsynchronously.tsx

```
import { FaPencil } from "react-icons/fa6";
export default function WorkingWithArraysAsynchronously() {
  const [todos, setTodos] = useState<any[]>([]);
  const editTodo = (todo: any) => {
    const updatedTodos = todos.map(
      (t) => t.id === todo.id ? { ...todo, editing: true } : t );
    setTodos(updatedTodos);
  };
  const updateTodo = async (todo: any) => {
    await client.updateTodo(todo);
    setTodos(todos.map((t) => (t.id === todo.id ? todo : t)));
  };
  return (
    <div id="wd-asynchronous-arrays">
      <h3>Working with Arrays Asynchronously</h3>
      <ListGroup>
        {todos.map((todo) => (
          <ListGroupItem key={todo.id}>
            <FaPencil onClick={() => editTodo(todo)} className="text-primary float-end me-2 mt-1" />
            <input type="checkbox" defaultChecked={todo.completed} className="form-check-input me-2 float-start"
              onChange={(e) => updateTodo({ ...todo, completed: e.target.checked }) } />
              {!todo.editing ? ( todo.title ) : (
                <FormControl className="w-50 float-start" defaultValue={todo.title}
                  onKeyDown={(e) => {
                    if (e.key === "Enter") {
                      updateTodo({ ...todo, editing: false });
                    }
                  }}
                  onChange={(e) =>
                    updateTodo({ ...todo, title: e.target.value })
                  }
                />
              )}
    ...
);}
```

5.2.6.4 Handling Errors

The exercises so far have been very optimistic when interacting with the server, but it is good practice to handle edge cases and the unforeseen. In this section we're going to add error handling to some of the routes and user interface. For instance, the exercise below throws exceptions if the items being deleted or updated don't actually exist. Errors are reported by the server as status codes, where 404 is the infamous NOT FOUND error. Additionally a JSON object can be sent back as part of the response that can be used by user interfaces to better inform the user of what went wrong.

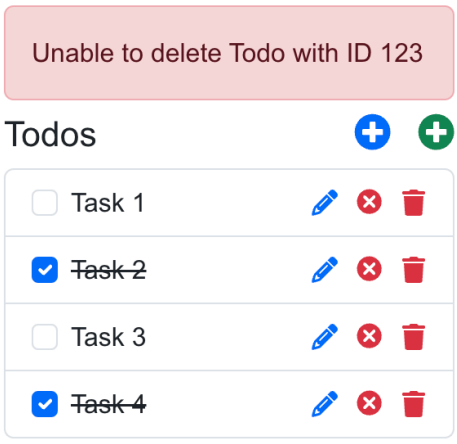
Lab5/WorkingWithArrays.js

```
  const deleteTodo = (req, res) => {
    const { id } = req.params;
    const todoIndex = todos.findIndex((t) => t.id === parseInt(id));
    if (todoIndex === -1) {
      res.status(404).json({ message: `Unable to delete Todo with ID ${id}` });
      return;
    }
    todos.splice(todoIndex, 1);
    res.sendStatus(200);
  });
  const updateTodo = (req, res) => {
    const { id } = req.params;
    const todoIndex = todos.findIndex((t) => t.id === parseInt(id));
    if (todoIndex === -1) {
      res.status(404).json({ message: `Unable to update Todo with ID ${id}` });
      return;
    }
    todos = todos.map((t) => { ... });
    res.sendStatus(200);
  }
```

In the user interface we can ***catch*** the errors by wrapping the request in a ***try***/***catch*** clause as shown below. If the request fails with a HTTP error response, then the body of the ***try*** block is aborted and the body of the ***catch*** clause executes instead. The exercise below declares a ***errorMessage*** state variable that we populate with the error from the server if an error occurs. The error is rendered as an alert box as shown here on the right. To test, remove an item using the http://localhost:4000/lab5/todos/:id/delete route and then try to update or delete the same item using the user interface. Confirm you get an error if you try to delete or update a ***todo*** that does not exist.

app/labs/lab5/WorkingWithArrays.tsx

```
export default function WorkingWithArraysAsynchronously() {
  const [errorMessage, setErrorMessage] = useState(null);
  const updateTodo = async (todo: any) => {
    try {
      await client.updateTodo(todo);
      setTodos(todos.map((t) => (t.id === todo.id ? todo : t)));
    } catch (error: any) {
      setErrorMessage(error.response.data.message);
    }
  };
  const deleteTodo = async (todo: any) => {
    try {
      await client.deleteTodo(todo);
      const newTodos = todos.filter((t) => t.id !== todo.id);
      setTodos(newTodos);
    } catch (error: any) {
      console.log(error);
      setErrorMessage(error.response.data.message);
    }  };  ...
  return (
    <div id="wd-asynchronous-arrays">
      <h3>Working with Arrays Asynchronously</h3>
      {errorMessage && (<div id="wd-todo-error-message" className="alert alert-danger mb-2 mt-2">{errorMessage}</div>)}
      ...
    </div>
);}
```



5.3 Next.js Server Routes

Next.js API routes provide a powerful, built-in way to create server-side endpoints directly within a Next.js application, eliminating the need for a separate backend server in many cases. Defined by simply creating files inside the app/api/ directory, each file automatically becomes an API endpoint. For example, a file at ***app/api/hello/route.ts*** becomes accessible at ***/api/hello***. These routes support all standard HTTP methods (***GET***, ***POST***, ***PUT***, ***DELETE***, etc.) and give

developers full control over the request and response objects, making it straightforward to handle form submissions, authentication, database operations, third-party API proxying, or any custom server logic — all while staying inside the same Next.js project.

The example below demonstrates a trivial hello world route that responds with a simple JSON object when accessing the URL ***http://localhost:3000/api/lab5/hello***.

app/api/lab5/hello/route.ts

```
import { NextRequest, NextResponse } from "next/server";

export async function GET(request: NextRequest) {
 return NextResponse.json({ message: "Hello from Lab 5 API!" });
}
```

5.3.1 Next.js Calculator Web API

To practice Next.js API routes in the App Router, let's implement a simple yet practical Web API that defines a basic calculator endpoint. Create a route in ***app/api/lab5/calculator/route.ts***, that implements a server-side calculator accessible via a ***GET*** request at the URL path ***/api/lab5/calculator***. The code reads query parameters from the request URL (***a***, ***b***, and ***operation***), performs basic arithmetic (***addition*** or ***subtraction***), validates the inputs, and returns a JSON response with the operands, operation, and computed result.

A request to ***/api/lab5/calculator?a=10&b=5&operation=add*** produces the response ***{ "a": 10, "b": 5, "operation": "add", "result": 15 }***. Invalid inputs like missing numbers or an unrecognized operation return error messages with the appropriate status code. Extend the calculator to calculate ***multiplication*** and ***division***.

app/api/lab5/calculator/route.ts

```
import { NextRequest, NextResponse } from "next/server";

export async function GET(request: NextRequest) {
 const { searchParams } = new URL(request.url);
 const a = parseFloat(searchParams.get("a") ?? "");
 const b = parseFloat(searchParams.get("b") ?? "");
 const operation = searchParams.get("operation");

 if (isNaN(a) || isNaN(b)) {
   return NextResponse.json({ error: "Invalid numbers" }, { status: 400 });
 }

 let result: number;
 switch (operation) {
   case "add":
     result = a + b;
     break;
   case "subtract":
     result = a - b;
     break;
   default:
     return NextResponse.json({ error: "Invalid operation" }, { status: 400 });
 }

 return NextResponse.json({ a, b, operation, result });
}
```

The component below implements a client user interface to the server calculator Web API. The React client component ("***use client***") provides an interactive frontend that integrates with the server through the ***/api/lab5/calculator*** endpoint we built earlier, demonstrating a complete full-stack workflow of Next.js API routes. The component uses React's ***useState*** hooks to manage inputs for two numbers (***a*** and ***b***), the selected ***operation***, and dynamic states for the result and any

errors. It triggers a simple ***fetch*** call to the ***/api/lab5/calculator*** route whenever the user switches operations via the dropdown or clicks the ***Calculate*** button. Parameters are encoded as query parameters. JSON responses are parsed and displayed formatted output such as "3 + 5 = 8" in a read-only input field.

app/labs/lab5/CalculatorNextWebApiClient.tsx

```
"use client";
import { useState } from "react";
import { Alert } from "react-bootstrap";

const OPERATIONS = [
 { label: "+", value: "add" },
 { label: "-", value: "subtract" },
 { label: "×", value: "multiply" },
 { label: "÷", value: "divide" },
];

export default function CalculatorNextWebApiClient() {
 const [a, setA] = useState("");
 const [b, setB] = useState("");
 const [operation, setOperation] = useState("add");
 const [result, setResult] = useState<number | null>(null);
 const [error, setError] = useState<string | null>(null);
 const [loading, setLoading] = useState(false);

 const calculate = async (op?: string) => {
   setOperation(op ?? operation);
   setResult(null);
   setError(null);
   setLoading(true);
   try {
     const res = await fetch(
       `/api/lab5/calculator?a=${encodeURIComponent(a)}&b=${encodeURIComponent(b)}&operation=${op ?? operation}`,
     );
     const data = await res.json();
     if (!res.ok) {
       setError(data.error ?? "Unknown error");
     } else {
       setResult(data.result);
     }
   } catch {
     setError("Failed to reach the server");
   } finally {
     setLoading(false);
   }
 };

 const operationSymbol =
   OPERATIONS.find((op) => op.value === operation)?.label ?? "";

 return (
   <div className="p-4">
     <h2>Calculator (Next.js Web API)</h2>
     <label htmlFor="a-input" className="w-25"> A: </label>
     <input type="number" value={a} placeholder="A" id="a-input" className="form-control w-25 d-inline"
       onChange={(e) => setA(e.target.value)} />
     <br />

     <label htmlFor="b-input" className="w-25"> B: </label>
     <input type="number" value={b} placeholder="B" id="b-input" className="form-control w-25 d-inline"
       onChange={(e) => setB(e.target.value)} />
     <br />

     <label htmlFor="operation-select" className="w-25"> Operation: </label>
     <select id="operation-select" value={operation} className="form-select w-25 d-inline"
       onChange={(e) => { calculate(e.target.value); }}>
       {OPERATIONS.map((op) => (
         <option key={op.value} value={op.value}>
           {op.label}
         </option>
       ))}
     </select>
```

```
      <br />

      <button className="btn btn-primary mt-3 w-50" disabled={loading || a === "" || b === ""}
        onClick={() => calculate(operation)}>
        {loading ? "Calculating..." : "Calculate"}
      </button>
      <br />

      <label className="w-25">Result:</label>
      {result !== null && (
        <input readOnly value={`${a} ${operationSymbol} ${b} = ${result}`} className="form-control w-25 d-inline" />)}
      {error && (
        <Alert variant="danger" className="mt-3 mb-0">
          {error}
        </Alert>
      )}
    </div>
);}
```

5.4 Implementing the Kambaz Node.js HTTP Server

Kambaz is currently implemented entirely as a React application. Although various CRUD operations have been implemented to create, read, update, and delete courses and modules, these changes are not permanent and are lost when the browser is refreshed. To make the changes permanent, it is necessary to integrate the React user interface with a server that can access resources such as the file system, operating system, network, and database. In this section, server routes will be implemented to integrate the user interface with the server. In the next chapter, changes will be stored permanently in a MongoDB non-relational database.

5.4.1 Migrating the "Database" to the Server

Previous chapters, declared a ***Database*** component to consolidate all data files into a single access point. Ideally, the data should reside on the server side or within a dedicated database. In this chapter we are going to move the data to the server, with a transition to a database in the subsequent chapter. Begin by creating a folder named ***Kambaz*** at the root of the Node.js project. Inside the ***Kambaz*** folder, create a ***Database*** directory and copy all JSON files from the React project. Then, change the file extensions of the JSON files to JavaScript, for example, rename ***users.json*** to ***users.js*** and ***courses.json*** to ***courses.js***. At the top of each newly converted JavaScript file, include an export default statement, as shown below.

kambaz/database/courses.js ***s***

```
export default [
  { _id: "RS101", name: "Rocket Propulsion", number: "RS4550", startDate: "2023-01-10",
    endDate: "2023-05-15", department: "D123", credits: 4, description: "..." }, ...
];
```

Do the same for all the JSON files and update the import statements in the ***Database*** component as shown below. Use the same data files from previous chapters. Feel free to modify the data in the files to customize the content or meet requirements in this chapter. Ignore unnecessary data files.

kambaz/database/index.js ***s***

```
import courses from "./courses.js";
import modules from "./modules.js";
import assignments from "./assignments.js";
import users from "./users.js";
import grades from "./grades.js";
import enrollments from "./enrollments.js";
export default { courses, modules, assignments, users, grades, enrollments };
```

5.4.2 Integrating the Account Screens with the Server with RESTful Web APIs

The ***Data Access Object*** (***DAO***) design pattern organizes data access by grouping it based on data types or collections. The following ***Users/dao.js*** file implements various CRUD operations for handling the ***users*** array in the ***Database***. Later sections in the chapter will create additional ***DAO***s for each of the data arrays: courses, modules, etc.

kambaz/users/dao.js *s*

```
import { v4 as uuidv4 } from "uuid";
export default function UsersDao(db) {
 let { users } = db;
 const createUser = (user) => {
   const newUser = { ...user, _id: uuidv4() };
   users = [...users, newUser];
   return newUser;
 };
 const findAllUsers = () => users;
 const findUserById = (userId) => users.find((user) => user._id === userId);
 const findUserByUsername = (username) => users.find((user) => user.username === username);
 const findUserByCredentials = (username, password) =>
   users.find((user) => user.username === username && user.password === password);
 const updateUser = (userId, user) => (users = users.map((u) => (u._id === userId ? user : u)));
 const deleteUser = (userId) => (users = users.filter((u) => u._id !== userId));
 return {
   createUser, findAllUsers, findUserById, findUserByUsername, findUserByCredentials, updateUser, deleteUser };
}
```

Like in the React project, install the ***uuid*** library in the Node.js project as shown below to generate unique identifiers when creating new instances of courses, modules, and other object instances.

```
npm install uuid
```

5.4.2.1 Integrating the React Sign In Screen with a RESTful Web API

DAOs provide an interface between an application and low-level database access, offering a high-level API to the rest of the application while abstracting the details and idiosyncrasies of using a particular database vendor. Similarly, routes create an interface between the HTTP network layer and the JavaScript object and function layer by transforming a stream of bits from a network connection request into a set of objects, maps, and function event handlers that are part of the client/server architecture in a multi-tiered application.

The Node.js server implements uses routes to integrate with the user interface and implements DAOs to communicate with the ***Database***. The server functions between these two layers, which is why it is often called the ***middle tier*** in a ***multi-tiered*** application. The following routes expose the database operations through a RESTful API, and the implementation of each function will be covered in the following sections. This chapter uses the ***Database*** component implemented in ***Database/index.ts***. Later chapters will refactor this by using an actual database.

kambaz/users/routes.js *s*

```
import UsersDao from "./dao.js";
let currentUser = null;
export default function UserRoutes(app, db) {
 const dao = UsersDao(db);
  const createUser = (req, res) => { };
  const deleteUser = (req, res) => { };
  const findAllUsers = (req, res) => { };
  const findUserById = (req, res) => { };
  const updateUser = (req, res) => { };
  const signup = (req, res) => { };
  const signin = (req, res) => { };
  const signout = (req, res) => { };
  const profile = (req, res) => { };
```

```
  app.post("/api/users", createUser);
  app.get("/api/users", findAllUsers);
  app.get("/api/users/:userId", findUserById);
  app.put("/api/users/:userId", updateUser);
  app.delete("/api/users/:userId", deleteUser);
  app.post("/api/users/signup", signup);
  app.post("/api/users/signin", signin);
  app.post("/api/users/signout", signout);
  app.post("/api/users/profile", profile);
}
```

Import and configure the routes in ***index.js*** as shown below. Pass a reference to the database to each of the routes.

index.js ***s***

```
import express from "express";
...
import db from "./(kambaz)/database/index.js";
import UserRoutes from "./(kambaz)/users/routes.js";

const app = express();
UserRoutes(app, db);
...
app.listen(process.env.PORT || 4000);
```

Routes implement ***RESTful Web API***s that clients can use to integrate with server functionality. The route implemented below extracts properties ***username*** and ***password*** from the request's body and passes them to the ***findUserByCredentials*** function implemented by the DAO. The resulting user is stored in the server variable ***currentUser*** to remember the logged in user. The user is then sent to the client in the response. Later sections will add error handling in case the user is not found in the database.

kambaz/users/routes.js ***s***

```
import UsersDao from "./dao.js";
let currentUser = null;
export default function UserRoutes(app, db) {
 const dao = UsersDao(db);
  ...
  const signin = (req, res) => {
    const { username, password } = req.body;
    currentUser = dao.findUserByCredentials(username, password);
    res.json(currentUser);
  };
  app.post("/api/users/signin", signin);
  ...
}
```

In the React user interface, under ***kambaz/account***, implement the ***client*** shown below to integrate with the user routes implemented in the server. The client function ***signin*** shown below posts a ***credentials*** object containing the ***username*** and ***password*** expected by the server. If the credentials are found, the response should contain the logged in user.

app/(kambaz)/account/client.ts ***c***

```
import axios from "axios";
export const HTTP_SERVER = process.env.NEXT_PUBLIC_HTTP_SERVER;
export const USERS_API = `${HTTP_SERVER}/api/users`;

export const signin = async (credentials: any) => {
  const response = await axios.post( `${USERS_API}/signin`, credentials );
  return response.data;
};
```

Implement a ***Sign in*** screen users can use to authenticate with the application. The following component declares state variable ***credentials*** to edit the ***username*** and ***password***. Clicking the ***Sign in*** button posts the ***credentials*** to the server using the ***client.signin*** function. When the server responds successfully, the currently logged in user is stored in the user reducer and navigate to the ***Profile*** screen implemented in a later section.

app/(kambaz)/account/signin/page.tsx ***c***

```
import * as client from "../client";
export default function Signin() {
  const [credentials, setCredentials] = useState<any>({});
  const dispatch = useDispatch();
  const signin = async () => {
    const user = await client.signin(credentials);
    if (!user) return;
    dispatch(setCurrentUser(user));
    redirect("/dashboard");
  };
  return ( ... );
}
```

5.4.2.2 Integrating the React Sign Up Screen with a RESTful Web API

The DAO implements functions ***createUser*** and ***findUserByUsername*** as shown below. The ***createUser*** DAO function accepts a user object from the user interface and then inserts the user into the ***Database***. The ***findUserByUsername*** accepts a ***username*** from the user interface and finds the user with the matching ***username***.

Users/dao.js

```
import { v4 as uuidv4 } from "uuid";
...
const createUser = (user) => {
  const newUser = { ...user, _id: uuidv4() };
  users = [...users, newUser];
  return newUser;
};
const findUserByUsername = (username) => users.find((user) => user.username === username);
```

The DAO functions are used to implement the ***sign up*** operation for users to sign up to the application. The ***signup*** route expects a user object with at least the properties ***username*** and ***password***. The DAO's ***findUserByUsername*** is called to check if a user with that username already exists. If such a user is found a 400 error status is returned along with an error message for display in the user interface. If the username is not already taken the user is inserted into the database and stored in the ***currentUser*** server variable. The response includes the newly created user. The ***signup*** route is mapped to the ***api/users/signup*** path.

Users/routes.js

```
import UsersDao from "./dao.js";
let currentUser = null;
export default function UserRoutes(app, db) {
  const dao = UsersDao(db);
  ...
  const signup = (req, res) => {
    const user = dao.findUserByUsername(req.body.username);
    if (user) {
      res.status(400).json(
        { message: "Username already in use" });
      return;
    }
    currentUser = dao.createUser(req.body);
    res.json(currentUser);
  };
  app.post("/api/users/signup", signup);
  ...
```

```
}
```

Meanwhile in the React user interface Web app, implement a ***signup*** client that posts the new user to the Web API as shown below.

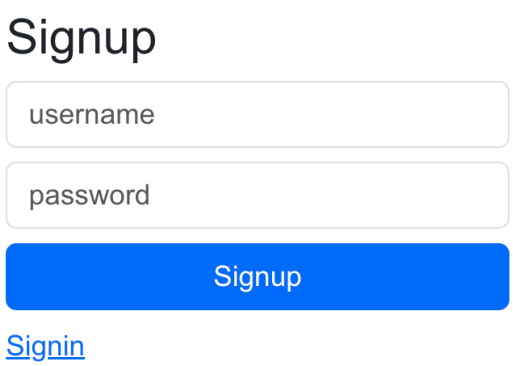
app/(kambaz)/account/client.ts

```
...
export const signup = async (user: any) => {
  const response = await axios.post(`${USERS_API}/signup`, user);
  return response.data;
};
...
```

If not already done so, implement a ***Sign up*** screen component that users can use to type their username and password, and post the credentials to the server for signing up. If the sign up is successful, navigate to the ***Profile*** screen. In the ***Account*** component, update the ***signup*** route to display the new ***Sign up*** screen. In the ***Sign in*** screen, create a ***Link*** to navigate to the ***Sign up*** screen. Confirm that you can signup with a new username and password. Style the ***Sign up*** screen so it looks as shown below on the right. Confirm navigates to profile and shows new user. Confirm you can navigate

app/(kambaz)/account/signup/page.tsx

```
"use client";
import Link from "next/link";
import { redirect } from "next/navigation";
import { setCurrentUser } from "../reducer";
import { useDispatch } from "react-redux";
import { useState } from "react";
import { FormControl, Button } from "react-bootstrap";
import * as client from "../client";

export default function Signup() {
  const [user, setUser] = useState<any>({});
  const dispatch = useDispatch();
  const signup = async () => {
    const currentUser = await client.signup(user);
    dispatch(setCurrentUser(currentUser));
    redirect("/profile");
  };
  return (
    <div className="wd-signup-screen">
      <h1>Sign up</h1>
      <FormControl value={user.username} onChange={(e) => setUser({ ...user, username: e.target.value })}
             className="wd-username b-2" placeholder="username" />
      <FormControl value={user.password} onChange={(e) => setUser({ ...user, password: e.target.value })}
             className="wd-password mb-2" placeholder="password" type="password"/>
      <button onClick={signup} className="wd-signup-btn btn btn-primary mb-2 w-100"> Sign up </button><br />
      <Link href="/account/signin" className="wd-signin-link">Sign in</Link>
    </div>
);}
```



5.4.2.3 Integrating the React Profile Screen with a RESTful Web API

In the ***User***'s ***DAO***, implement ***updateUser*** as shown below to update a single document by first identifying it by its primary key, and then updating the matching fields in the ***user*** parameter.

Users/dao.js ***s***

```
const updateUser = (userId, user) => (users = users.map((u) => (u._id === userId ? user : u)));
```

In the ***User***'s ***routes***, make the ***DAO*** function available as a RESTful Web API as shown below. Map a route that accepts a user's primary key as a path parameter, passes the ID and request body to the DAO function and responds with the status.

Users/routes.js ***s***

```
...
export default function UserRoutes(app, db) {
  ...
  const updateUser = (req, res) => {
    const userId = req.params.userId;
    const userUpdates = req.body;
    dao.updateUser(userId, userUpdates);
    currentUser = dao.findUserById(userId);
    res.json(currentUser);
  };
  ...
  app.put("/api/users/:userId", updateUser);
}
```

In the React client application, add client function ***updateUser*** to send user updates to the server to be saved to the database.

app/(kambaz)/account/client.ts ***c***

```
export const updateUser = async (user: any) => {
  const response = await axios.put(`${USERS_API}/${user._id}`, user);
  return response.data;
};
```

In the ***Profile*** screen implement the ***updateProfile*** event handler as shown below to update the profile on the server. Add an ***Update*** button that invokes the new handler. Confirm that the profile changed by logging out and then logging back in.

app/(kambaz)/account/profile/page.tsx ***c***

```
...
import * as client from "../client";
import { RootState } from "../../store";
export default function Profile() {
  const [profile, setProfile] = useState<any>({});
  const dispatch = useDispatch();
  const { currentUser } = useSelector((state: RootState) => state.accountReducer);
  const updateProfile = async () => {
    const updatedProfile = await client.updateUser(profile);
    dispatch(setCurrentUser(updatedProfile));
  };
  ...
  return (
    <div id="wd-profile-screen">
      <h3>Profile</h3>
      {profile && (
        ...
        <div>
          <button onClick={updateProfile} className="btn btn-primary w-100 mb-2"> Update </button>
          <button ...> Sign out </button>
        </div>
      )}
    </div>
);}
```

5.4.2.4 Retrieving the Profile from the Server

When a successful sign in occurs, the account information is stored in a server variable called ***currentUser***. The variable retains the signed-in user information as long as the server is running. The ***Sign in*** screen copies currentUser from the server into the ***currentUser*** state variable in the reducer and then navigates to the Profile screen. If the browser reloads,

the ***currentUser*** state variable is cleared and the user is logged out. To address this, the browser must check whether someone is already logged in from the server and, if so, update the copy in the reducer. Create a route on the server to provide access to ***currentUser*** as shown below.

Users/routes.js

```
let currentUser = null;
export default function UserRoutes(app, db) {
  ...
  const signin = async (, res) => { ... };
  const profile = async (req, res) => {
    res.json(currentUser);
  };
  ...
  app.post("/api/users/signin",  signin);
  app.post("/api/users/profile", profile);
}
```

Then in the React Web app, implement a function to retrieve the ***account*** information from the server route implemented above as shown below.

app/(kambaz)/account/client.ts

```
import axios from "axios";
export const USERS_API = process.env.NEXT_PUBLIC_HTTP_SERVER;
export const signin = async (user) => { ... };
export const profile = async () => {
  const response = await axios.post(`${USERS_API}/profile`);
  return response.data;
};
```

Create a new ***Session*** component that fetches the ***current user*** from the server and stores it in the store so that the rest of the application can have access to the ***current user***.

src/account/Session.tsx

```
import * as client from "./client";
import { useEffect, useState } from "react";
import { setCurrentUser } from "./reducer";
import { useDispatch } from "react-redux";
export default function Session({ children }: { children: any }) {
  const [pending, setPending] = useState(true);
  const dispatch = useDispatch();
  const fetchProfile = async () => {
    try {
      const currentUser = await client.profile();
      dispatch(setCurrentUser(currentUser));
    } catch (err: any) {
      console.error(err);
    }
    setPending(false);
  };
  useEffect(() => {
    fetchProfile();
  }, []);
  if (!pending) {
    return children;
  }
}
```

Wrap the Kambaz application with ***Session*** component so that it renders before all other components to check if anyone is signed in. Once it figures out either way, it'll store the result in the store and let the rest of the components render.

Confirm that it works by signing in, and then from the Profile screen, reload the browser. Make sure the user information still renders correctly.

app/(kambaz)/layout.ts

```
...
"use client";
import Session from "./account/Session";
export default function KambazLayout(...) {
  ...
  return (
    <Provider store={store}>
     <Session>
        <div>
        ...
        </div>
      </Session>
    </Provider>
);}
```

5.4.2.5 Integrating Signout with a RESTful Web API

Implement a route for users to signout that resets the ***currentUser*** to null in the server as shown below.

Users/routes.js

```
let currentUser = null;
export default function UserRoutes(app, db) {
  ...
  const signout = (req, res) => {
    currentUser = null;
    res.sendStatus(200);
  };
  app.post("/api/users/signout", signout);
  ...
}
export default UserRoutes;
```

In the React user interface Web application, add a client function that can post to the ***signout*** route.

app/(kambaz)/account/client.ts

```
export const signout = async () => {
  const response = await axios.post(`${USERS_API}/signout`);
  return response.data;
};
```

In the ***Profile*** screen refactor the ***signout*** function to invoke the ***signout*** client function and then navigates to the ***Sign in*** screen. Confirm that you can signout and navigate to the ***Sign in*** screen.

app/(kambaz)/account/profile/page.tsx

```
  ...
  const signout = async () => {
    await client.signout();
    dispatch(setCurrentUser(null));
    redirect("/account/signin");
  };
  ...
  <button onClick={signout} className="wd-signout-btn btn btn-danger w-100">
    Sign out
  </button>
  ...
```

5.4.3 Supporting Multiple User Sessions

The user authentication implemented so far is simple but supports only one signed-in user at a time. Web applications typically support multiple users signed in simultaneously. This section describes how to add session handling to the Node.js server to allow multiple users to be signed in at the same time.

5.4.3.1 Installing and Configuring Server Sessions

First, it is necessary to narrow down who is allowed to authenticate. Configure ***CORS*** to support cookies and restrict network access to come only from the React application as shown below.

index.js

```
const app = express();
app.use(
 cors({
   credentials: true,                                          // support cookies
   origin: process.env.CLIENT_URL || "http://localhost:3000",  // restrict cross origin resource
 })                                                            // sharing to the react application
);
app.use(express.json());
const port = process.env.PORT || 4000;
```

In the Node.js project, install the ***express-session*** and ***dotenv*** libraries to maintain application sessions and read configurations from environment variables on the server.

```
$ npm  install  express-session
$ npm  install  dotenv
```

In a new ***.env*** file at the root of the project, declare the following environment variables.

.env

```
SERVER_ENV=development
CLIENT_URL=http://localhost:3000
SERVER_URL=http://localhost:4000
SESSION_SECRET=super secret session phrase
```

In ***index.js***, import the ***dotenv*** library to determine whether the application is running in the development environment, and configure the session as shown below. Make sure to configure sessions ***after*** configuring ***cors***. Note: the following configuration has been tested on ***Google's Chrome*** and ***Apple's Safari*** browsers.

index.js

```
import "dotenv/config";                                        // import the new dotenv library
import session from "express-session";                         // to read .env file
const app = express();
app.use(
  cors({
    credentials: true,
    origin: process.env.CLIENT_URL || "http://localhost:3000",  // use different front end URL in dev
  })                                                           // and in production
);
const sessionOptions = {                                       // default session options
  secret: process.env.SESSION_SECRET || "kambaz",
  resave: false,
  saveUninitialized: false,
};
```

```
if (process.env.SERVER_ENV !== "development") {                    // in production
  sessionOptions.proxy = true;                                     // turn on proxy support
  sessionOptions.cookie = {                                        // configure cookies for remote server
    sameSite: "none",
    secure: true,
    domain: process.env.SERVER_URL,
  };
}
app.use(session(sessionOptions));
app.use(express.json());                                           // make sure this comes AFTER configuring cors
...                                                                // and session, but BEFORE all the routes
```

The ***signup*** route retrieves the ***username*** from the request body. If a user with that username already exists, an error is returned. Otherwise, create the new user and store it in the session's ***currentUser*** property to remember that this new user is now the currently logged-in user.

kambaz/users/routes.js

```
let currentUser = null;
...
export default function UserRoutes(app, db) {
  ...
  const signup = (req, res) => {
    const user = dao.findUserByUsername(req.body.username);
    if (user) {
      res.status(400).json({ message: "Username already taken" });
      return;
    }
    const currentUser = dao.createUser(req.body);
    req.session["currentUser"] = currentUser;
    res.json(currentUser);
  };
```

An existing user can identify themselves by providing credentials. The ***signin*** route below looks up the user by their credentials, stores it in ***currentUser*** session, and responds with the user if they exist. Otherwise responds with an error.

kambaz/users/routes.js

```
  const signin = (req, res) => {
    const { username, password } = req.body;
    const currentUser = dao.findUserByCredentials(username, password);
    if (currentUser) {
      req.session["currentUser"] = currentUser;
      res.json(currentUser);
    } else {
      res.status(401).json({ message: "Unable to login. Try again later." });
    }
  };
```

If a user has already signed in, the ***currentUser*** can be retrieved from the session by using the ***profile*** route as shown below. If there is no ***currentUser***, an error is returned.

kambaz/users/routes.js

```
const profile = (req, res) => {
    const currentUser = req.session["currentUser"];
    if (!currentUser) {
      res.sendStatus(401);
      return;
    }
    res.json(currentUser);
  };
```

Users can be signed out by destroying the session.

kambaz/users/routes.js

```
  const signout = (req, res) => {
    req.session.destroy();
    res.sendStatus(200);
  };
```

If a user updates their profile, then the session must be kept in synch.

kambaz/users/routes.js

```
  const updateUser = (req, res) => {
    const userId = req.params.userId;
    const userUpdates = req.body;
    dao.updateUser(userId, userUpdates);
    const currentUser = dao.findUserById(userId);
    req.session["currentUser"] = currentUser;
    res.json(currentUser);
  };
```

5.4.3.2 Configuring Axios to Support Server Sessions

By default ***axios*** does not support cookies. To configure ***axios*** to include cookies in requests, use the ***axios.create()*** to create an instance of the library that includes cookies for credentials as shown below. Then replace all occurrences of the ***axios*** library with this new version ***axiosWithCredentials***.

app/(kambaz)/account/client.ts

```
import axios from "axios";
const axiosWithCredentials = axios.create({ withCredentials: true });
export const HTTP_SERVER = process.env.NEXT_PUBLIC_HTTP_SERVER;
export const USERS_API = `${HTTP_SERVER}/api/users`;
export const signin = async (credentials: any) => {
  const response = await axiosWithCredentials.post(`${USERS_API}/signin`, credentials);
  return response.data;
};
export const profile = async () => {
  const response = await axiosWithCredentials.post(`${USERS_API}/profile`);
  return response.data;
};
export const signup = async (user: any) => {
  const response = await axiosWithCredentials.post(`${USERS_API}/signup`, user);
  return response.data;
};
export const signout = async () => {
  const response = await axiosWithCredentials.post(`${USERS_API}/signout`);
  return response.data;
};
export const updateUser = async (user: any) => {
  const response = await axiosWithCredentials.put(`${USERS_API}/${user._id}`, user);
  return response.data;
};
```

5.4.5 Creating a RESTful Web API for Courses

Previous chapters implemented CRUD operations to create, read, update and delete courses in the Kambaz Dashboard. These changes were transient and were lost when users refreshed the browser. This section demonstrates implementing a RESTful Web API to integrate the ***Dashboard*** and ***Courses*** screen with the server. The API will migrate the CRUD operations from the user interface to the server where they belong.

5.4.5.1 Retrieving Courses

Now that the ***Database*** has been moved to the server, it must be made available to the React client application through a Web ***API*** (***Application Programming Interface***). The exercises below make the courses accessible at http://localhost:4000/api/courses for the React user interface to integrate. First implement a DAO to retrieve all courses from the ***Database***.

kambaz/courses/dao.js

```
import { v4 as uuidv4 } from "uuid";
export default function CoursesDao(db) {
  function findAllCourses() {
    return db.courses;
  }
  return { findAllCourses };
}
```

Use the ***DAO*** to implement a route that retrieves all the courses.

kambaz/courses/routes.js

```
import CoursesDao from "./dao.js";
export default function CourseRoutes(app, db) {
  const dao = CoursesDao(db);
  const findAllCourses = (req, res) => {
    const courses = dao.findAllCourses();
    res.send(courses);
  }
  app.get("/api/courses", findAllCourses);
}
```

In ***index.js*** import the new routes and pass a reference to the express module to the routes. Make sure to work ***AFTER*** the ***cors***, ***session***, and ***json use*** statements. Point your browser to http://localhost:4000/api/courses and confirm the server responds with an array of courses.

index.js

```
import express from "express";
import Lab5 from "./Lab5/index.js";
import UserRoutes from "./(kambaz)/users/routes.js";
import CourseRoutes from "./(kambaz)/courses/routes.js";
import db from "./(kambaz)/database/index.js";
import cors from "cors";
const app = express();
app.use(cors(...));      // make sure cors is configured BEFORE session
app.use(session(...));   // make sure session is configure BEFORE express.json
app.use(express.json()); // make sure express.json is configure BEFORE all routes
UserRoutes(app, db);
CourseRoutes(app, db);
Lab5(app);
app.listen(4000);
```

Since the ***Dashboard*** displays courses a user is enrolled in, implement ***findCoursesForEnrolledUser*** as shown below to retrieve courses the current user is enrolled in.

kambaz/courses/dao.js

```
...
function findCoursesForEnrolledUser(userId) {
  const { courses, enrollments } = db;
  const enrolledCourses = courses.filter((course) =>
    enrollments.some((enrollment) => enrollment.user === userId && enrollment.course === course._id));
```

```
  return enrolledCourses;
}
...
return {
  findAllCourses,
  findCoursesForEnrolledUser,
};
```

Since enrolled courses are retrieved within the context of the currently logged in user, implement the following route to retrieve the courses in the user routes.

kambaz/courses/routes.js

```
import CoursesDao from "./dao.js";
export default function CourseRoutes(app, db) {
  const dao = CoursesDao(db);
  ...
  const findCoursesForEnrolledUser = (req, res) => {
    let { userId } = req.params;
    if (userId === "current") {
      const currentUser = req.session["currentUser"];
      if (!currentUser) {
        res.sendStatus(401);
        return;
      }
      userId = currentUser._id;
    }
    const courses = dao.findCoursesForEnrolledUser(userId);
    res.json(courses);
  };
  app.get("/api/users/:userId/courses", findCoursesForEnrolledUser);
  ...
}
```

Back in the user interface, create a ***client.ts*** file under the ***kambaz/courses*** that implements all the course related communication between the user interface and the server. Start by implementing the ***fetchAllCourses*** client function as shown below.

app/(kambaz)/courses/client.ts

```
import axios from "axios";
const HTTP_SERVER = process.env.NEXT_PUBLIC_HTTP_SERVER;

const COURSES_API = `${HTTP_SERVER}/api/courses`;
export const fetchAllCourses = async () => {
  const { data } = await axios.get(COURSES_API);
  return data;
};
```

But the ***Dashboard*** only displays the courses the current logged in user is enrolled in, so implement ***findMyCourses*** that retrieves the current user's courses using the new ***findCoursesForEnrolledUser*** end point.

app/(kambaz)/courses/client.ts

```
import axios from "axios";
const axiosWithCredentials = axios.create({ withCredentials: true });
const HTTP_SERVER = process.env.NEXT_PUBLIC_HTTP_SERVER;
const COURSES_API = `${HTTP_SERVER}/api/courses`;
const USERS_API = `${HTTP_SERVER}/api/users`;

export const fetchAllCourses = async () => {
  const { data } = await axios.get(COURSES_API);
  return data;
};
```

```
export const findMyCourses = async () => {
  const { data } = await axiosWithCredentials.get(`${USERS_API}/current/courses`);
  return data;
};
...
```

In the ***Dashboard*** use ***useEffect*** to fetch the courses from the server on component load and update the ***courses*** in the reducer using the ***setCourse*** reducer function. Use the ***currentUser*** in the ***accountReducer*** as a dependency so that if a different user logs in, the courses will be reloaded from the server. Remove ***Database*** references from the user interface since we don't need it anymore. Also initialize the ***courses*** state variable as empty since we won't have the database anymore. Also remove the filtering of courses by enrollments since the server is already doing that. Restart the server and user interface to confirm that the ***Dashboard*** renders the courses the current user is enrolled in. Sign in as different users and confirm only the courses the user is enrolled in display in the ***Dashboard***.

app/(kambaz)/dashboard/page.tsx

```
import * as db from "./database";
import * as client from "../courses/client";
import { addNewCourse, deleteCourse, updateCourse, setCourses } from "../courses/reducer";
...
export default function Dashboard({ ... }) {
 const { courses } = useSelector((state: RootState) => state.coursesReducer);
 const { enrollments } = db;
 const { currentUser } = useSelector((state: RootState) => state.accountReducer);
 const dispatch = useDispatch();
 const [course, setCourse] = useState<any>({ ... });
  const fetchCourses = async () => {
    try {
      const courses = await client.findMyCourses();
      dispatch(setCourses(courses));
    } catch (error) {
      console.error(error);
    }
  };
  useEffect(() => {
    fetchCourses();
  }, [currentUser]);
  ...
  return (
    <div id="wd-dashboard">
      ...
      {courses
        .filter((course) => enrollments.some((enrollment) =>
                enrollment.user === currentUser._id &&
                enrollment.course === course._id))
        .map((course) => ( ... ))}
      ...
    </div>
);}
```

5.4.5.2 Creating New Courses

Implement a route that creates a new course and adds it to the ***Database***. The new course is passed in the HTTP body from the client and is appended to the end of the courses array in the ***Database***. The new course is given a new unique identifier and sent back to the client in the response.

kambaz/courses/dao.js

```
import { v4 as uuidv4 } from "uuid";
...
function createCourse(course) {
  const newCourse = { ...course, _id: uuidv4() };
  db.courses = [...db.courses, newCourse];
  return newCourse;
```

```
}
...
return { findAllCourses, findCoursesForEnrolledUser, createCourse };
```

When a course is created, it needs to be associated with the creator. In a new ***Enrollments/dao.js*** file, implement ***enrollUserInCourse*** to enroll, or associate, a user to a course.

kambaz/enrollments/dao.js

```
import { v4 as uuidv4 } from "uuid";
export default function EnrollmentsDao(db) {
  function enrollUserInCourse(userId, courseId) {
    const { enrollments } = db;
    enrollments.push({ _id: uuidv4(), user: userId, course: courseId });
  }
  return { enrollUserInCourse };
}
```

In the ***Courses***'s routes, implement ***createCourse*** as shown below to create a new course and then enroll the ***currentUser*** in the new course. Respond with the ***newCourse*** so it can be rendered in the user interface.

kambaz/courses/routes.js

```
import CoursesDao from "./dao.js";
import EnrollmentsDao from "../enrollments/dao.js";
export default function CourseRoutes(app, db) {
 const dao = CoursesDao(db);
 const enrollmentsDao = EnrollmentsDao(db);
  const createCourse = (req, res) => {
    const currentUser = req.session["currentUser"];
    const newCourse = dao.createCourse(req.body);
    enrollmentsDao.enrollUserInCourse(currentUser._id, newCourse._id);
    res.json(newCourse);
  };
  app.post("/api/users/current/courses", createCourse);
  ...
}
```

In the course's ***client.ts***, add a ***createCourse*** client function that ***posts*** a new course to the server and returns the response's data which should be the brand new course created in the server.

app/(kambaz)/courses/client.ts

```
export const createCourse = async (course: any) => {
  const { data } = await axiosWithCredentials.post(`${USERS_API}/current/courses`, course);
  return data;
};
```

In the ***Dashboard*** screen, add an ***onAddCourse*** event handler that ***posts*** the new course to the server, and then appends the new course from the response to the end of the ***courses*** state variable. Refactor the ***Add*** button to use the new ***onAddCourse*** event handler. Confirm that creating a new course updates the user interface with the added course.

app/(kambaz)/dashboard/page.tsx

```
...
import * as client from "./courses/client";
export default function Dashboard() {
  ...
  const onAddNewCourse = async () => {
    const newCourse = await client.createCourse(course);
    dispatch(setCourses([ ...courses, newCourse ]));
```

```
  };
  ...
  useEffect(() => {...}, []);
  return (
    ...
       <button onClick={onAddNewCourse} className="btn btn-primary float-end" id="wd-add-new-course-click" >
         Add
       </button>
    ...
);}
```

5.4.5.3 Deleting a Course

Implement a route that removes a course and all enrollments associated with the course. First implement a ***deleteCourse*** DAO function that filters the course by its ID and then filters out all enrollments by the course's ID as shown below.

kambaz/courses/dao.js

```
export default function CoursesDao(db) {
  ...
  function deleteCourse(courseId) {
    const { courses, enrollments } = db;
    db.courses = courses.filter((course) => course._id !== courseId);
    db.enrollments = enrollments.filter(
      (enrollment) => enrollment.course !== courseId
  );
  ...
  return { findAllCourses, findCoursesForEnrolledUser, createCourse, deleteCourse };
}
```

In the Course's routes, implement a ***delete*** route that parses the course's ID from the URL and uses the ***deleteCourse*** DAO function as shown below. Remember to group callback functions at the top and the route declarations at the bottom.

kambaz/courses/routes.js

```
import CoursesDao from "./dao.js";
import EnrollmentsDao from "../enrollments/dao.js";
export default function CourseRoutes(app, db) {
  const dao = CoursesDao(db);
  const enrollmentsDao = EnrollmentsDao(db);
  const findAllCourses = (req, res) => { ... }
  const deleteCourse = (req, res) => {
    const { courseId } = req.params;
    const status = dao.deleteCourse(courseId);
    res.send(status);
  }
  app.delete("/api/courses/:courseId", deleteCourse);
  app.get("/api/courses", findAllCourses);
}
```

In the course's ***client.ts***, add a ***deleteCourse*** client function that ***deletes*** an existing course from the server and returns the status response from the server.

app/(kambaz)/courses/client.ts

```
export const deleteCourse = async (id: string) => {
  const { data } = await axios.delete(`${COURSES_API}/${id}`);
  return data;
};
```

In the ***Dashboard*** page, implement ***onDeleteCourse*** event handler to use the ***client*** to ***delete*** courses from the server and then filter out the course from the redux ***courses*** state variable. Reimplement the ***Delete*** button to use the new ***onDeleteCourse*** event handler as shown below. Confirm clicking ***Delete*** actually removes the course from the ***Dashboard***.

app/(kambaz)/dashboard/page.tsx

```
import * as client from "./courses/client";
export default function Dashboard() {
  ...
  const onDeleteCourse = async (courseId: string) => {
    const status = await client.deleteCourse(courseId);
    dispatch(setCourses(courses.filter((course) => course._id !== courseId)));
  };
  ...
    <button className="btn btn-danger"
            onClick={(event) => {
              event.preventDefault();
              onDeleteCourse(course._id);
            }} >
      Delete
    </button>
  ...
}
```

5.4.5.4 Updating a Course

In the Course's DAO, implement ***updateCourse*** function to update a course in the ***Database***. First lookup the course by its ID and then apply the updates to the course as shown below.

kambaz/courses/dao.js

```
...
function updateCourse(courseId, courseUpdates) {
  const { courses } = db;
  const course = courses.find((course) => course._id === courseId);
  Object.assign(course, courseUpdates);
  return course;
}
...
return { findAllCourses, findCoursesForEnrolledUser, createCourse, deleteCourse, updateCourse };
```

In the Course's routes, implement a ***put*** route that parses the ***id*** of course as a path parameter and updates uses the ***updateCourse*** DAO function to update the corresponding course with the updates in HTTP request body. If the update is successful, respond with status 204.

kambaz/courses/routes.js

```
import CoursesDao from "./dao.js";
import EnrollmentsDao from "../enrollments/dao.js";
export default function CourseRoutes(app, db) {
  ...
  const updateCourse = (req, res) => {
    const { courseId } = req.params;
    const courseUpdates = req.body;
    const status = dao.updateCourse(courseId, courseUpdates);
    res.send(status);
  }
  app.put("/api/courses/:courseId", updateCourse);
  ...
}
```

In the course's ***client.ts***, add a ***updateCourse*** client function that ***updates*** an existing course in the server and returns the status response from the server.

app/(kambaz)/courses/client.ts

```
export const updateCourse = async (course: any) => {
```

```
  const { data } = await axios.put(`${COURSES_API}/${course._id}`, course);
  return data;
};
```

In the ***Dashboard*** screen, implement an ***onUpdateCourse*** event handler so that it uses the ***client*** to ***put*** the updated course to the server and then swaps out the old corresponding course with the new version in the ***course*** state variable. Refactor the ***Update*** button so that it uses the new ***onUpdateCourse*** event handler. Confirm that clicking the ***Update*** button actually updates the course in the ***Dashboard***.

app/(kambaz)/dashboard/page.tsx

```
import * as client from "./courses/client";
export default function Dashboard() {
  ...
  const onUpdateCourse = async () => {
    await client.updateCourse(course);
    dispatch(setCourses(courses.map((c) => {
        if (c._id === course._id) { return course; }
        else { return c; }
    })));};
  ...
  useEffect(() => {...}, []);
  return (
    ...
      <button onClick={onUpdateCourse} className="btn btn-secondary float-end" id="wd-update-course-click" >
        Update
      </button>
    ...
);}
```

5.4.6 Creating a RESTful Web API for Modules

Now let's do the same thing we did for the courses, but for the modules. We'll need routes that deal with the modules similar to the operations we implemented for the courses. We'll need to implement all the basic ***CRUD*** operations: ***create*** modules, ***read***/retrieve modules, ***update*** modules and ***delete*** modules. The main difference will be that modules exist with the context of a particular course. Each course has a different set of modules, so the routes will need to take into account the course ID for which the modules we are operating on.

5.4.6.1 Retrieving a Course's Modules

Create ***DAO*** for the ***Modules*** to implement module data access from the ***Database***. Start by implementing ***findModulesForCourse*** to retrieve a course's modules by its ID as shown below.

kambaz/modules/dao.js

```
export default function ModulesDao(db) {
 function findModulesForCourse(courseId) {
   const { modules } = db;
   return modules.filter((module) => module.course === courseId);
 }
 return {
   findModulesForCourse,
 };
}
```

Create a new routes file for the ***Module*** with a route to retrieve the modules for a course by its ID encoded in the path. Parse the course ID from the path and then use the module's DAO ***findModulesForCourse*** function to retrieve the modules for that course. In index.js, import the new route file and pass it a reference to the ***app*** and ***db***.

kambaz/modules/routes.js

```
import ModulesDao from "../modules/dao.js";
export default function ModulesRoutes(app, db) {
  const dao = ModulesDao(db);
  const findModulesForCourse = (req, res) => {
    const { courseId } = req.params;
    const modules = dao.findModulesForCourse(courseId);
    res.json(modules);
  }
  app.get("/api/courses/:courseId/modules", findModulesForCourse);
}
```

In the Course's ***client.js*** file in the React project, create ***findModulesForCourse*** to integrate the user interface with the server as shown below. Implement ***findModulesForCourse*** function shown below which retrieves the modules for a given course.

app/(kambaz)/courses/client.ts

```
import axios from "axios";
const HTTP_SERVER = process.env.NEXT_PUBLIC_HTTP_SERVER;
const COURSES_API = `${HTTP_SERVER}/api/courses`;
export const findModulesForCourse = async (courseId: string) => {
  const response = await axios
    .get(`${COURSES_API}/${courseId}/modules`);
  return response.data;
};
...
```

Update the modules reducer implemented in earlier chapters as shown below. Remove dependencies from the ***Database*** since we've moved it to the server. Empty the ***modules*** state variable since we'll be populating it with the modules we retrieve from the server using the ***findModulesForCourse*** function. Add a ***setModules*** reducer function so we can populate the ***modules*** state variable when we retrieve the modules from the server.

app/(kambaz)/courses/[cid]/modules/reducer.tsx

```
import db from "../database";
const initialState = {
  modules: [],
};
const modulesSlice = createSlice({
  name: "modules",
  initialState,
  reducers: {
    setModules: (state, action) => {
      state.modules = action.payload;
    },
    addModule: (...) => {...},
    deleteModule: (...) => {...},
    updateModule: (...) => {...},
    editModule: (...) => {...},
  },
});
export const { addModule, deleteModule, updateModule, editModule, setModules } = modulesSlice.actions;
export default modulesSlice.reducer;
```

In the ***Modules*** component, import the new ***setModules*** reducer function and the new ***findModulesForCourse*** client function. Using a ***useEffect*** function, invoke the ***findModulesForCourse*** client function and dispatch the modules from the server to the reducer with the ***setModules*** function as shown below. Remove the filter since modules are already filtered on the server. Confirm that navigating to a course populates the corresponding modules.

app/(kambaz)/courses/[cid]/modules/page.tsx

```
import { setModules, addModule, editModule, updateModule, deleteModule } from "./reducer";
import { useState, useEffect } from "react";
import * as client from "../../client";
export default function Modules() {
  const { modules } = useSelector((state: any) => state.modulesReducer);
  const { cid } = useParams();
  const dispatch = useDispatch();
  const fetchModules = async () => {
    const modules = await client.findModulesForCourse(cid as string);
    dispatch(setModules(modules));
  };
  useEffect(() => {
    fetchModules();
  }, []);

 return (
   <div>
     ...
     <ListGroup id="wd-modules" className="rounded-0">
       {modules
         .filter((module: any) => module.course === cid)
         .map((module: any) => ( ... ))}
     </ListGroup>
   </div>
);}
```

5.4.6.2 Creating Modules for a Course

To create a new module in the ***Database***, implement ***createModule*** in the Module's DAO as shown below. The function accepts the new module as a parameter, sets its primary key and then appends the new module to the ***Database's*** module array. Add the new ***createModule*** to the map returned by ***ModulesDao***.

kambaz/modules/dao.js

```
import { v4 as uuidv4 } from "uuid";

function createModule(module) {
  const newModule = { ...module, _id: uuidv4() };
  db.modules = [...db.modules, newModule];
  return newModule;
}
function findModulesForCourse(courseId) { ... }
```

In the ***Module***'s routes, implement a ***post*** Web API for the user interface to integrate to when creating a new module for a given course. Parse the course's ID from the path and the new module from the request's body. Set the new module's course's property to the course's ID so that the module knows what course it belongs to. Use the module's DAO's ***createModule*** function to create the new module and then respond with the new module.

kambaz/modules/routes.js

```
import ModulesDao from "../modules/dao.js";
export default function ModulesRoutes(app, db) {
  const dao = ModulesDao(db);
  const findModulesForCourse = async (req, res) => { ... }
  const createModuleForCourse = (req, res) => {
    const { courseId } = req.params;
    const module = {
      ...req.body,
      course: courseId,
    };
    const newModule = dao.createModule(module);
    res.send(newModule);
  }
  app.post("/api/courses/:courseId/modules", createModuleForCourse);
  app.get("/api/courses/:courseId/modules", findModulesForCourse);
  ...
```

```
}
```

In the user interface, implement a ***createModuleForCourse*** client function that posts new modules from the user interface to the server as shown below. Encode the course's ID in the URL so the server knows what course the module belongs to.

app/(kambaz)/courses/client.ts

```
import axios from "axios";
const HTTP_SERVER = process.env.NEXT_PUBLIC_HTTP_SERVER;
const COURSES_API = `${HTTP_SERVER}/api/courses`;
export const createModuleForCourse = async (courseId: string, module: any) => {
  const response = await axios.post(
    `${COURSES_API}/${courseId}/modules`,
    module
  );
  return response.data;
};
export const findModulesForCourse = async (courseId: string) => { ... };
...
```

In the ***Modules*** screen, implement a ***onCreateModuleForCourse*** event handler that uses the new ***createModuleForCourse*** client function to send the module to the server and then dispatches the created module to the reducer so it's added to the reducer's ***modules*** state variable. Update the ***ModulesControls addModule*** attribute so that it uses the new ***onCreateModuleForCourse*** event handler. Confirm that new modules are created for the current course.

app/(kambaz)/courses/[cid]/modules/index.ts

```
import { addModule, editModule, updateModule, deleteModule, setModules } from "./reducer";
import { useSelector, useDispatch } from "react-redux";
import * as coursesClient from "../client";
export default function Modules() {
  const { cid } = useParams();
  const [moduleName, setModuleName] = useState("");
  const { modules } = useSelector((state: RootState) => state.modulesReducer);
  const dispatch = useDispatch();
  const onCreateModuleForCourse = async () => {
    if (!cid) return;
    const newModule = { name: moduleName, course: cid };
    const module = await client.createModuleForCourse(cid, newModule);
    dispatch(setModules([...modules, module]));
  };
  return (
    <div>
      <ModulesControls setModuleName={setModuleName} moduleName={moduleName} addModule={onCreateModuleForCourse} />
      ...
    </div> );}
```

5.4.6.3 Deleting a Module

In the ***Modules DAO***, implement ***deleteModule*** to remove a module from the ***Database*** by its ***ID*** as shown below. Add the new function to the return object.

kambaz/modules/dao.js

```
function deleteModule(moduleId) {
  const { modules } = db;
  db.modules = modules.filter((module) => module._id !== moduleId);
}
function createModule(module) { ... }
function findModulesForCourse(courseId) { ... }
```

In the ***Modules*** router file, implement a route that handles an ***HTTP DELETE*** to remove a module by its ID. Parse the module's ID from the path and use the ***DAO***'s ***deleteModule*** function to remove the module from the ***Database***.

kambaz/modules/routes.js

```
const deleteModule = (req, res) => {
  const { moduleId } = req.params;
  const status = dao.deleteModule(moduleId);
  res.send(status);
}
app.delete("/api/modules/:moduleId", deleteModule);
```

In the Courses's ***client*** file, implement the ***deleteModule*** function as shown below. Pass it the ***ID*** of the module to be removed, encode it in a URL, and sent it as an ***HTTP DELETE*** to the server.

app/(kambaz)/courses/client.ts

```
const MODULES_API = `${HTTP_SERVER}/api/modules`;
export const deleteModule = async (moduleId: string) => {
 const response = await axios.delete(`${MODULES_API}/${moduleId}`);
 return response.data;
};
```

In the ***Modules*** screen, use the ***client*** file and use it to implement the ***onRemoveModule*** event handler as shown below. Use the new ***deleteModule*** client function to remove the module from the server. If successful, filter the module from the modules array and dispatch the new list of modules to the reducer to remove the module from the modules state variable as well. In the ***ModuleControlsButtons*** component, update the ***deleteModule*** attribute to use the new ***onRemoveModule*** event handler. Confirm that clicking the trashcan of modules removes the modules. Refresh the screen to make sure that the modules is permanently deleted.

app/(kambaz)/courses/[cid]/modules/page.tsx

```
import { addModule, editModule, updateModule, deleteModule, setModules } from "./reducer";
import { useSelector, useDispatch } from "react-redux";
import * as client from "./client";
export default function Modules() {
  const { cid } = useParams();
  const [moduleName, setModuleName] = useState("");
  const { modules } = useSelector((state: RootState) => state.modulesReducer);
  const dispatch = useDispatch();
  const onRemoveModule = async (moduleId: string) => {
    await client.deleteModule(moduleId);
    dispatch(setModules(modules.filter((m: any) => m._id !== moduleId)));
  };
  ...
  return (
    <div>
      ...
      <ListGroup id="wd-modules" className="rounded-0">
        {modules.map((module: any) => (
              ...
              <ModuleControlButtons moduleId={module._id}
               deleteModule={(moduleId) => onRemoveModule(moduleId)}
               editModule={(moduleId) => dispatch(editModule(moduleId))} />
        ))}
      </ListGroup>
    </div>);}
```

5.4.6.4 Update Module

In the ***Module***'s ***DAO***, implement ***updateModule*** to update a module in the ***Database*** by its ***ID***. First lookup the module by its ***ID*** and then apply the updates to the module as shown below. Add the new ***updateModule*** to the return object.

kambaz/modules/dao.js

```
function updateModule(moduleId, moduleUpdates) {
  const { modules } = db;
  const module = modules.find((module) => module._id === moduleId);
  Object.assign(module, moduleUpdates);
  return module;
}
function deleteModule(moduleId) { ... }
function createModule(module) { ... }
function findModulesForCourse(courseId) { ... }
```

In the ***Module***'s ***routes*** file, implement an ***HTTP PUT*** request handler that parses the ***ID*** of the course from the ***URL*** and the module updates from the ***HTTP*** request body. Use the ***DAO***'s ***updateModule*** function to apply the updates to the module.

kambaz/modules/routes.js

```
const updateModule = async (req, res) => {
  const { moduleId } = req.params;
  const moduleUpdates = req.body;
  const status = await dao.updateModule(moduleId, moduleUpdates);
  res.send(status);
}
app.put("/api/modules/:moduleId", updateModule);
```

In the ***client*** file, implement the ***updateModule*** function as shown below. Pass it the module to be update. Encode the ***ID*** of the module in a URL, and sent the module updates in the body of an ***HTTP PUT*** request to the server.

app/(kambaz)/courses/client.ts

```
export const updateModule = async (module: any) => {
  const { data } = await axios.put(`${MODULES_API}/${module._id}`, module);
  return data;
};
```

In the ***Modules*** screen, implement a ***onUpdateModule*** event handler as shown below. Use the new ***updateModule*** client function to update the module in the server. If successful, dispatch the updated module to the reducer to update the module in the ***modules*** state variable as well. In the ***onKeyDown*** event handler, invoke the ***saveModule*** event handler. Confirm that updating the module in the user interface actually modifies the module on the server. Refresh the screen to make sure that the modules has been modified.

app/(kambaz)/courses/[cid]/modules/page.tsx

```
...
import { addModule, editModule, updateModule, deleteModule, setModules } from "./reducer";
import { useSelector, useDispatch } from "react-redux";
import * as client from "./client";
export default function Modules() {
  const { cid } = useParams();
  const [moduleName, setModuleName] = useState("");
  const { modules } = useSelector((state: RootState) => state.modulesReducer);
  const dispatch = useDispatch();
  const onUpdateModule = async (module: any) => {
    await client.updateModule(module);
    const newModules = modules.map((m: any) => m._id === module._id ? module : m );
    dispatch(setModules(newModules));
  };
  ...
  return (
    <div>
      ...
      <ListGroup id="wd-modules" className="rounded-0">
```

```
        {modules.map((module: any) => (
          ...
          {!module.editing && module.name}
          { module.editing && (
            <FormControl className="w-50 d-inline-block" value={module.name}/>
              onChange={(e) => dispatch(updateModule({ ...module, name: e.target.value })) }
              onKeyDown={(e) => {
                if (e.key === "Enter") {
                    onUpdateModule({ ...module, editing: false });
                }
              }}
            )}
      ))}
    </ListGroup>
  </div> );}
```

5.4.7 Assignments and Assignments Editor (On Your Own)

In your Node.js server application, implement routes for ***creating***, ***retrieving***, ***updating***, and ***deleting*** assignments. In the React Web application, create an ***assignment client*** file that uses ***axios*** to send ***POST***, ***GET***, ***PUT***, and ***DELETE HTTP*** requests to integrate the React application with the server application. In the React user interface, refactor the ***Assignments*** and ***AssignmentEditor*** screens implemented in earlier chapters to use the new ***client*** file to ***CRUD*** assignments. New assignments, updates to assignments, and deleted assignments should persist if the screens are refreshed as long as the server is running.

5.4.8 Enrollments (On Your Own)

In your Node.js server application, implement routes to support the ***Enrollments*** screen. Users should be able to enroll and unenroll from courses. In the React application, implement an enrollments client that uses ***axios*** to integrate with the routes in the server. Enrollments should persist as long as the server is running.

5.4.9 People Table (Optional)

In your Node.js server application, implement routes to support the People screen. Users should be able to see all users enrolled in the course. Faculty should be able to create, update, and delete users. In the React application, implement an users client that uses ***axios*** to integrate with the routes in the server. User changes should persist as long as the server is running.

5.5 Deploying RESTful Web Service APIs to a Public Remote Server

Up to this point you should have a working two tiered application with the first tier consisting of a front end React user interface application and the second tier consisting of a Node Express HTTP server application. In this section we're going to learn how to replicate this setup so that it can execute on remote servers. All development should be done in the local development environment on your personal development computer, and only when we're satisfied that all works fine locally, then we can make an effort at deploying the application on remote servers. The React Web application is already configured to deploy and run remotely on Vercel when you commit and push to the GitHub repository containing the source for the project. This section demonstrates how to configure the Node Express HTTP server project to deploy to a remote server hosted by ***Render*** or ***Heroku*** and then integrate the remote React Web application on Vercel to the Node Express server deployed and running on ***Render*** (or ***Heroku***).

5.5.1 Committing and Pushing the Node Server Source to Github

First create a local Git repository in the Node Express project by typing ***git init*** at the command line at the root of the project. It's ok if the repository was already initialized.

```
$ git  init
```

Configure the Git repository to disregard unnecessary files in the repository by listing them in ***.gitignore***. Create a new file called ***.gitignore*** if it does not already exist. Note the leading period (.) in front of the file name. The file should contain at least ***node_modules***, but should also contain any IDE specific files or directories. If using IntelliJ, include the ***.idea*** folder as shown below. Environment files such as ***.env*** and ***.env.development*** should also be included in ***.gitignore***.

.gitignore

```
node_modules
.env.development
.env
```

Use ***git add*** to add all the source code into the repository and commit with a simple comment.

```
$ git  add  .
$ git  commit  -m  "first commit"
```

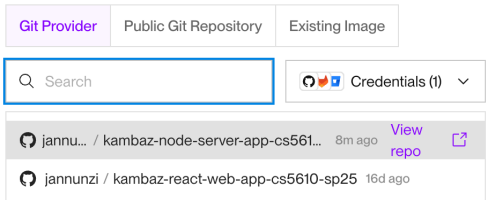
Head over to ***github.com*** and create a repository named ***kambaz-node-server-app***. Add this repository as the origin target using the git remote command. ***NOTE:*** make sure to use your github username instead of mine.

```
$ git remote add origin https://github.com/jannunzi/kambaz-node-server-app.git
```

Push the code in your local repository to the remote origin repository. Note that your branch might be called something else. Refresh the remote github repository and confirm the code is now available online.

```
$ git push -u origin main
```

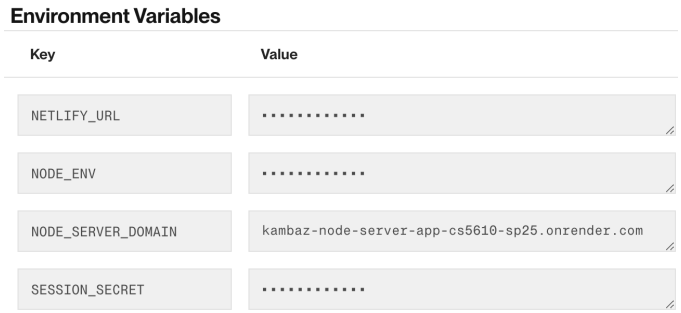
5.5.2 Deploying a Node Server Application to Render.com from Github



If you don't already have an account at ***render.com***, create a new account to deploy the Node server remotely. From the dashboard on the top right, select ***Add New*** and then ***Web Service***. In the ***New web service*** screen, in the ***Source Code*** option, select the ***Git Provider*** tab, search for the git repository created earlier and select it from the dropdown list. In the ***Name*** field, type the name of the application, e.g., use the same name as the github repository ***kambaz-node-server-app***, or something similar. In the ***Build Command*** field type ***npm install***. In the ***Start Command*** field type ***npm start*** or ***node index.js***. Under the ***Instance Type*** select ***Free***. In the ***Environment Variables*** section click ***Add Environment Variable*** to create all the environment variables in the ***.env*** file, but with values shown below.

Environment Variable	*Value*
SERVER_ENV	production
CLIENT_URL	https://kambaz-next-js-cs5610-sp25.Vercel.app
SERVER_URL	kambaz-node-server-app-cs5610-sp25.onrender.com
SESSION_SECRET	super secret session phrase

For the ***CLIENT_URL*** environment variable, use the URL of your React Web application deployed in Vercel, not mine as shown above. For ***SERVER_URL***, use the domain name in the ***Name*** field, but make sure it has the postfix ***.onrender.com*** as shown above. Note that after the application deploys, Render might modify the domain name slightly if the domain is already taken. You'll need to edit the environment variable so that its value is the actual domain name. Make sure that ***SERVER_URL*** does not contain ***http://*** or ***https://*** as a prefix.

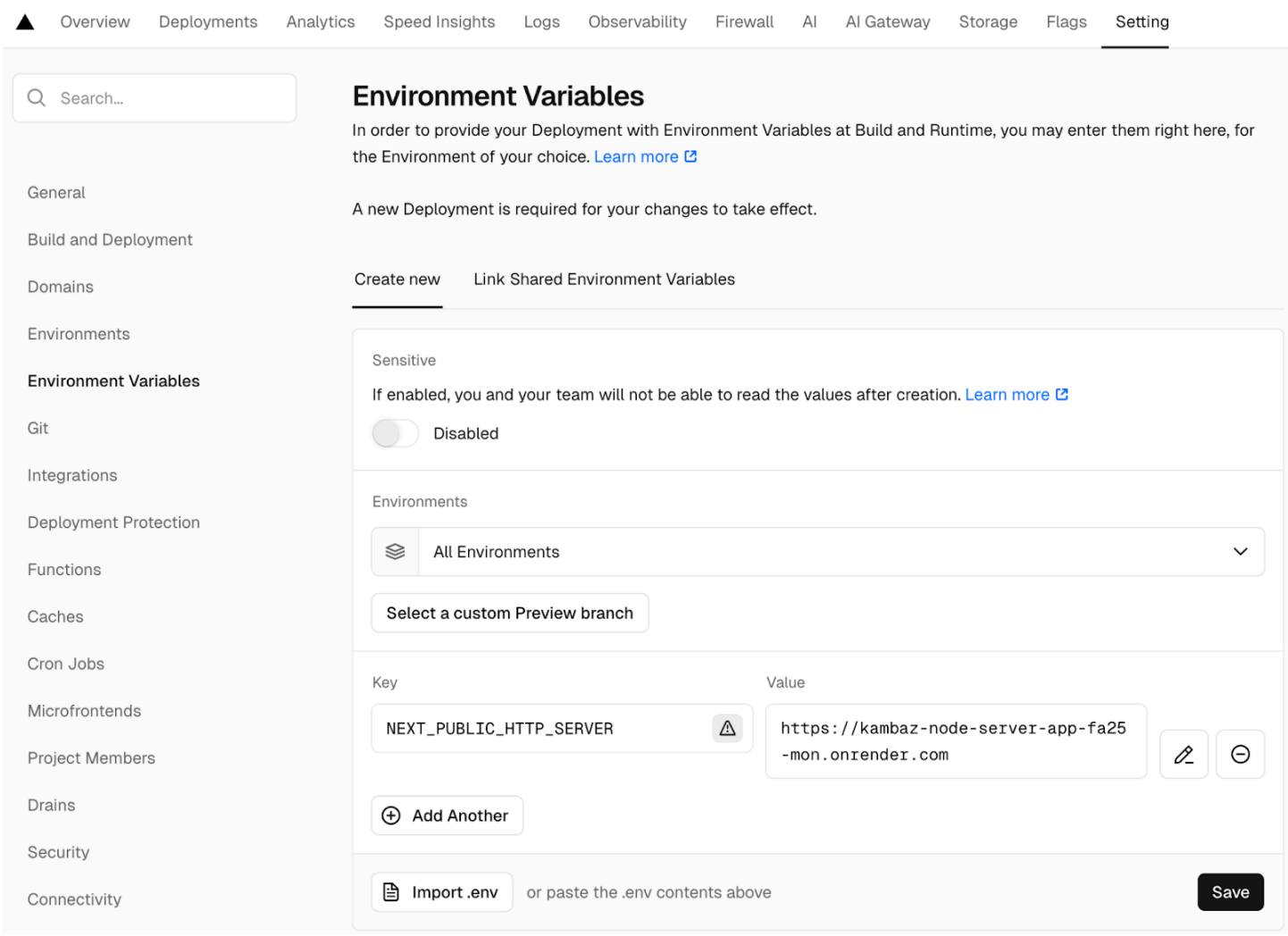


Click ***Deploy Web Service*** to deploy the server. In the deploy screen take a look at the logs. You can click on ***Maximize*** to see the logs better. Look for a ***Build successful*** message. You can click on ***Minimize*** to minimize the logs. If the deployment fails, fix whatever the logs complaint about, commit and push changes to GitHub, and try deploying again by selecting ***Deploy last commit*** from the ***Manual Deploy*** drop menu at the top right. If the deployment succeeds a URL appears at the top of the screen. Navigate to the URL, e.g., https://kambaz-node -server-app-cs5610-2035.onrender.com and confirm you get the same greeting you would get locally, e.g., ***Welcome to Full Stack Development!*** Also confirm you can get the array of courses from the remote API, e.g., https://kambaz-node-server-app-cs5610-2035.onrender.com/ api/courses. Finally, make sure you can get the list of modules for at least one of the courses, e.g., https://kambaz-node-server-app-cs5610-sp35.onrender.com/api/courses/RS101/modules. ***NOTE:*** the actual URL might be different based on the actual name you chose for the application.

If the name chosen was not unique, Render will modify the name of the domain so that it's unique. It might append a random string at the end of the name, e.g., https://kambaz-node-server-app-cs5610-2035-***qpwoeiru***.onrender.com. If this is the case, modify the ***SERVER_URL*** environment variable by clicking ***Environment*** on the left sidebar of the Render dashboard. Make sure all environment variables have the correct values and edit if necessary. To edit ***SERVER_URL***, click ***Edit*** and replace the value with the actual domain name as shown below. Do not include the protocol ***https://***, or extra slashes. Click on ***Save, rebuild, and deploy*** when done.

5.5.3 Configuring Remote Environment in Vercel

Now that the Node.js HTTP server is running remotely on Render.com, the React Web application running on Vercel needs to be configured to integrate with the server running on Render.com. Currently the React Web application is configured to connect to the local Node Express server, but when the Web application is running on ***Vercel*** it needs to connect to the remote Node Express server running on ***Render*** or ***Heroku***. Configure ***Vercel*** defining environment variable ***NEXT_PUBLIC_HTTP_SERVER*** so that it references the remote server running on ***Render*** or ***Heroku***.



To configure environment variables in Vercel, navigate to your project. In the ***Overview*** screen, navigate to the ***Deployment.*** In the ***Deployment Details*** screen, under the ***Environment*** label, navigate to ***Production***. In the ***Project Settings*** screen, navigate to ***Environment variables*** on the left sidebar. In the ***Environment Variables*** screen, in the ***Create new*** tab, in the ***Key*** input field, enter ***NEXT_PUBLIC_HTTP_SERVER***, and then in the ***Value*** field, copy and paste the root URL of the application running on Render or Heroku, e.g., ***https://kambaz-node-server-app-fa25-mon.onrender.com***. NOTE: make sure there's no trailing slash. Click ***Save*** and then ***Redeploy***. Note that here we do want the protocol ***https://***, but not in the ***SERVER_URL*** environment variable in Render.com. Redeploy the React application and confirm that the ***Dashboard*** renders the courses from the remote Node server and ***Modules*** still renders the modules for the selected course. Also confirm all the labs still work when running on Vercel. Using the ***Network*** tab in the ***Inspector*** in the ***Development Tools*** of the browser, make sure that none of the requests use ***http://localhost:3000***.

5.6 Conclusion

In this chapter we learned how to create HTTP servers using the Node.js JavaScript framework. We implemented RESTful services with the Express library and practiced sending, retrieving, modifying, and updating data using HTTP requests and responses. We then learned how to integrate React Web applications to HTTP servers implementing a client server architecture. In the next chapter we will add database support to the HTTP server so we can store data permanently.

5.7 Deliverables

As a deliverable, make sure you complete all the lab exercises, course, modules, and assignment routes on the server, as well as the client, and component refactoring on the React project. For both the React and Node repositories, all your work should be done in a branch called ***a5***. When done, add, commit and push both branches to their respective GitHub repositories. Deploy the new branches to ***Vercel*** and ***Render*** (or ***Heroku***) and confirm they integrate. All the lab exercises should work remotely just as well as locally. The Kambaz Dashboard should display the courses from the server as well as the modules, and assignments. In ***TOC.tsx***, add a link to the new GitHub repository and a link to the root of the server running on ***Render*** or ***Heroku***. As a deliverable in ***Canvas***, submit the URL to the ***a5*** branch deployment of your React application running on Vercel.